

```
In [8]: import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image
```

3-3 최적화 알고리즘

```
In [9]: Image('131.png')
```

Out[9]: **0. 데이터셋 소개**

Wild blueberry

[Wild blueberry](#) 야생 블루베리의 수확량을 시뮬레이션한 데이터셋입니다.

Features	Unit	Description
Row#		Index
clonesize	m2	The average blueberry clone size in the field
honeybee	bees/m2/min	Honeybee density in the field
bumbles	bees/m2/min	Bumblebee density in the field
andrena	bees/m2/min	Andrena bee density in the field
osmia	bees/m2/min	Osmia bee density in the field
MaxOfUpperTRange	°C	The highest record of the upper band daily air temperature during the bloom season
MinOfUpperTRange	°C	The lowest record of the upper band daily air temperature
AverageOfUpperTRange	°C	The average of the upper band daily air temperature
MaxOfLowerTRange	°C	The highest record of the lower band daily air temperature
MinOfLowerTRange	°C	The lowest record of the lower band daily air temperature
AverageOfLowerTRange	°C	The average of the lower band daily air temperature
RainingDays	Day	The total number of days during the bloom season, each of which has precipitation larger than zero
AverageRainingDays	Day	The average of raining days of the entire bloom season
fruitset		The amount of fruit per tree
fruitmass		The mass of a fruit
seeds		The mass of a seed
yield		The amount of yield

```
In [10]: df_berry = pd.read_csv('data/wild_blueberry.csv', index_col='Row#')
df_berry.head()
```

Out[10]:

	clonesize	honeybee	bumbles	andrena	osmia	MaxOfUpperTRange	MinOfUpperTRange	AverageOfUpperTRange	MaxOfLowerTF
Row#									
0	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	
1	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	
2	37.5	0.75	0.25	0.25	0.25	94.6	57.2	79.0	
3	37.5	0.75	0.25	0.25	0.25	94.6	57.2	79.0	
4	37.5	0.75	0.25	0.25	0.25	86.0	52.0	71.9	

In [11]:

Image('132.png')

Out[11]: 1. 경사 하강법(Gradient Descent)

아이디어

목적 함수의 최적값을 찾기 위해 목적 함수에 대한 파라미터(Parameter)의 경사(Gradient)를 이용합니다.

목적 함수의 최적값의 방향으로 파라미터(Parameter)를 경사도에 따라 이동시킵니다.

경사도를 구하고 이동을 반복하여 파라미터의 최적점을 도출합니다.

알고리즘

1. 파라미터($\hat{\beta}$) 초기화
2. 파라미터 경사도($\frac{\partial L}{\partial \hat{\beta}}$) 계산, L: 손실, 목적함수
3. 파라미터 업데이트

$$\hat{\beta} = \hat{\beta} - \text{Learning rating} \times \frac{\partial L}{\partial \hat{\beta}},$$

Learning rating : 경사도 반영율

4. 종료 조건 미충족 시 2번 부터 반복

종료 조건: 업데이트 반복횟수(epochs), 성능 개선 정도, ...

In [12]: `Image('133.png')`

Out[12]: **[Ex.1]**

$$y = X + 5 + e, e \sim N(0, 1)$$

관계를 지닌 데이터 포인트 100개를 만듭니다.

경사하강법의 동작 원리를 파악하기 위해, 기울기와 절편을 경사 하강법을 이용한 선형 회귀 모델 학습합니다.

여기서, 회귀 모델의 손실 함수(Loss Function)를 평균제곱오차(Mean Squared Error)를 이용하여

회귀 계수와 손실 함수 경사도를 구합니다.

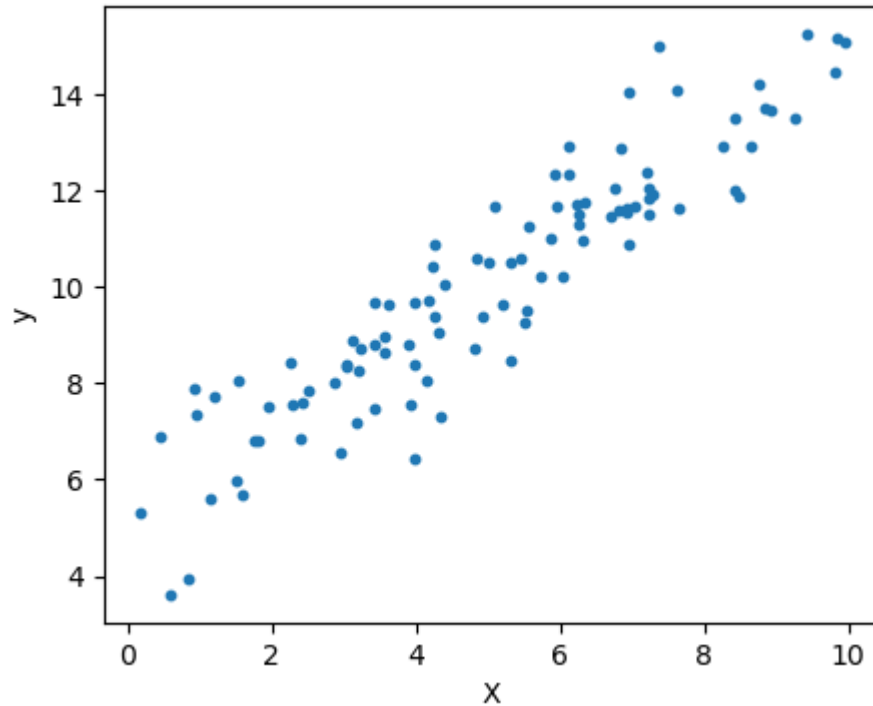
$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\hat{y} = X\hat{\beta}, r = y - \hat{y}$$

$$\frac{dL}{d\hat{\beta}} = -\frac{2}{n} X^T r$$

```
In [13]: # 단순 선형 회귀 데이터셋을 만듭니다.

# 종속변수 (y) 독립변수(X) 간의 기울기와 절편입니다.
beta_1, beta_0 = 1, 5
# 동일한 실행 결과를 얻게하기 위해 랜덤 시드값을 고정시킵니다.
np.random.seed(123)
# 기울기는 beta_1, 절편은 beta_0이고 오차가 표준정규분포인
# 임의의 100개의 데이터 포인트를 생성합니다.
X = np.random.random(size=100) * 10
y = X * beta_1 + beta_0 + np.random.normal(size=100)
plt.figure(figsize=(5, 4))
plt.scatter(X, y, s=10)
plt.gca().set_xlabel('X')
plt.gca().set_ylabel('y')
plt.show()
```



In [14]: `'''`
``np.vstack()`` 함수는 주어진 배열들을 수직 방향(위에서 아래로)으로
 쌓아서 새로운 배열을 생성합니다. 수직 방향으로 쌓이기 때문에
 배열들은 동일한 열 수를 가져야 합니다.
 이 함수는 배열을 쌓는 데 주로 사용되며,
 특히 데이터를 행렬 형태로 결합하거나 새로운 행을 추가할 때 유용합니다.

예를 들어, 두 개의 1차원 배열을 수직으로 쌓아서
 2차원 배열을 만들 수 있습니다.
 다음은 ``np.vstack()`` 함수를 사용하여
 두 개의 1차원 배열을 수직으로 쌓은 예시입니다.

```
array1 = np.array([1, 2, 3])
array2 = np.array([4, 5, 6])

result = np.vstack([array1, array2])
print(result)
```

위 코드는 다음과 같은 결과를 출력합니다.

```
[[1 2 3]
 [4 5 6]]
```

또한, `np.vstack()` 함수를 사용하여 여러 개의 배열을 한 번에 수직으로 쌓을 수도 있습니다. 이 경우에는 쌓고자 하는 배열을 리스트로 전달하면 됩니다. 모든 배열은 동일한 열 수를 가져야 합니다.

- ```
'''
...
1. `np.ones_like(X)`를 사용하여 X와 같은 크기의 배열을 생성하고,
 그 값은 모두 1로 채웁니다.
 이렇게 생성된 배열은 절편(intercept)을 나타내는데 사용됩니다.
2. `np.vstack([np.ones_like(X), X])`를 사용하여
 절편을 포함한 설명 변수 행렬을 생성합니다.
 이 때, `np.ones_like(X)`로 생성된 1로 채워진 배열과
 X를 수직으로 쌓아 올립니다.
3. `.T`를 사용하여 행렬의 전치(transpose)를 수행합니다.
 이렇게 하면 설명 변수가 열로 나열된 행렬이 됩니다.
4. `[:5]`를 사용하여 생성된 행렬의 첫 5개의 행을 출력합니다.
'''
```

결과적으로는, 절편을 포함한 설명 변수 행렬이 생성되고,  
그 중 첫 5개의 행이 출력됩니다.  
이 행렬은 다음과 같은 형태를 갖습니다:

```
[[1. , X_1],
 [1. , X_2],
 [1. , X_3],
 [1. , X_4],
 [1. , X_5]]
```

여기서 각 행은 하나의 데이터 포인트를 나타내며,  
첫 번째 열은 절편을 나타내고  
두 번째 열은 해당 데이터 포인트의 설명 변수(X)를 나타냅니다.

```
'''
회귀 모델의 절편을 따로 구하지 않고,
회귀 계수와 같이 구할 수 있도록 상수항을 추가합니다.
'''
```

```
X_add_const = np.vstack([np.ones_like(X), X]).T
X_add_const[:5]
```

```
Out[14]: array([[1. , 6.96469186],
 [1. , 2.86139335],
 [1. , 2.26851454],
 [1. , 5.51314769],
 [1. , 7.1946897]])
```

```
In [15]: '''
코드는 경사 하강법을 사용하여 선형 회귀 모델의 파라미터를
학습하는 과정을 나타냅니다.

1. `beta_hat`을 [-1, 1] 범위에서 무작위로 선택한
 두 개의 숫자로 초기화합니다.
'''

파라미터를 초기화 합니다.
beta_hat = np.random.uniform(-1, 1, size=2)
beta_hat
```

```
Out[15]: array([0.69106599, -0.6261925])
```

```
In [16]: Image('134.png')
```

```
Out[16]: 경사도를 구하는 부분만 가져옵니다.
```

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\hat{y} = X\hat{\beta}, r = y - \hat{y}$$

$$\frac{dL}{d\hat{\beta}} = -\frac{2}{n} X^T r$$

```
In [17]: '''
2. `epochs`는 파라미터 업데이트를 반복할 횟수를
 나타내는 변수입니다. 여기서는 500번의 반복을 수행합니다.
```

3. ``learning_rate``는 학습률을 나타내는 변수입니다.  
새로운 가중치 값을 계산할 때 사용됩니다.
4. ``n``은 데이터셋의 샘플 수를 나타내는 변수입니다.
5. ``losses``는 각 반복에서의 손실(`loss`)을  
저장하기 위한 리스트입니다.
6. ``beta_hats``는 일정한 간격으로 모수를 저장하기 위한  
리스트입니다. 이는 특정 간격마다 모수를 저장하고  
추이를 확인하기 위해 사용됩니다.
7. ``for`` 루프는 주어진 에폭(`epoch`)만큼 반복합니다.
8. 각 반복에서는 잔차(`residuals`)를 계산합니다.  
잔차는 실제 값(`y`)과 모델의  
예측 값(`np.dot(X_add_const, beta_hat)`)의  
차이를 나타냅니다.
9. 현재 손실(`loss`)을 계산하고, ``losses`` 리스트에 추가합니다.
10. 모수를 일정한 간격(여기서는 100번의 반복마다) 저장하고자  
할 때마다 모수를 ``beta_hats`` 리스트에 추가합니다.
11. 기울기를 계산하여 경사 하강법을 통해  
모수를 업데이트합니다.  
이를 통해 손실을 최소화하는 방향으로  
모델의 파라미터를 조정합니다.
12. 반복이 끝나면 최종 모수를  
``beta_hats`` 리스트에 추가합니다.

이렇게 함으로써 경사 하강법을 사용하여  
선형 회귀 모델의 파라미터를 학습하는 과정이 완료됩니다.  
'''

*# 파라미터 업데이트 루틴의 반복 횟수입니다.*

`epochs = 500`

*# 파라미터 업데이트 비율입니다.*

`learning_rate = 0.01`

`n = X_add_const.shape[0]`

`losses = list()`

`beta_hats = list()`

`for i in range(epochs):`

`residuals = y - np.dot(X_add_const, beta_hat)`

`losses.append(np.mean(residuals ** 2))`

*# 100 epoch 마다 모수를 기록합니다.*

`if i % 100 == 0:`

```

 beta_hats.append(beta_hat.copy())
 beta_hat_grad \
 = -2 * np.dot(X_add_const.T, residuals) / n
 beta_hat -= beta_hat_grad * learning_rate
 # 최종 모수를 남깁니다.
 beta_hats.append(beta_hat.copy())

```

In [18]:

```
'''
코드는 경사 하강법의 진행에 따라
모델의 변화를 시각화합니다.

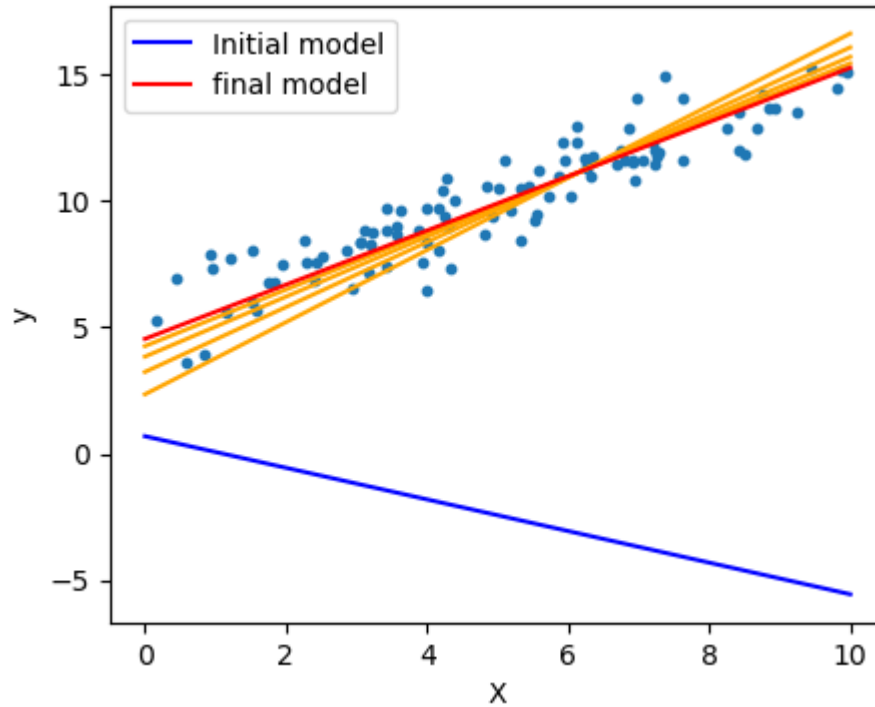
1. `plt.figure(figsize=(5, 4))`를 사용하여
 새로운 시각화를 시작합니다.
 그림의 크기는 5x4로 설정됩니다.
2. `plt.scatter(X, y, s=10)`를 사용하여 데이터 포인트를
 산점도로 플롯합니다.
 이전에 생성한 X와 y 데이터를 사용하여 플롯합니다.
3. 초기 모델의 직선을 그립니다.
 초기 모델은 `beta_hats` 리스트의 첫 번째 요소로부터
 얻은 모수를 사용합니다.
 이 초기 모델은 파란색으로 표시됩니다.
4. `for` 루프를 사용하여 각 반복에서의 모델의
 직선을 그립니다. 여기서는 각 반복에서 저장된 모수를
 사용하여 모델의 직선을 그립니다.
 각 반복에서의 모델은 주황색으로 표시됩니다.
5. 최종 모델의 직선을 그립니다. 최종 모델은 `beta_hats`
 리스트의 마지막 요소로부터 얻은 모수를 사용합니다.
 이 최종 모델은 빨간색으로 표시됩니다.
6. `plt.gca().set_xlabel('X')`와
 `plt.gca().set_ylabel('y')`를 사용하여
 x축과 y축의 레이블을 설정합니다.
7. `plt.legend()`를 사용하여 그림에 범례를 추가합니다.
8. `plt.show()`를 호출하여 그림을 출력합니다.
'''

```

이를 통해 경사 하강법의 각 단계에서 모델이  
어떻게 변하는지를 시각적으로 확인할 수 있습니다.  
처음에는 초기 모델이 파란색 직선으로 표시되고,  
반복이 진행됨에 따라 모델이 주황색 직선으로  
변화하다가 최종적으로 빨간색 직선으로  
수렴하는 것을 볼 수 있습니다.

```
'''

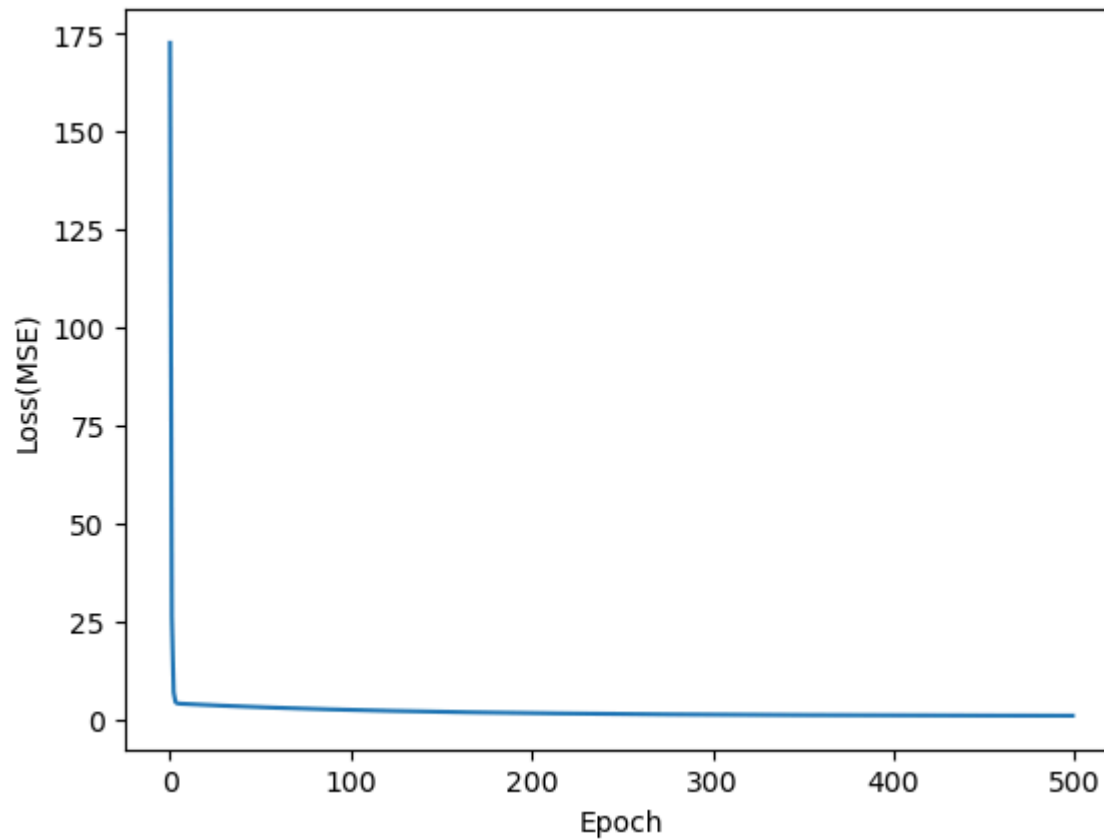
경사하강법의 진행됨에 따라 모델을 시각화 해봅니다.
plt.figure(figsize=(5, 4))
plt.scatter(X, y, s=10)
beta_hat = beta_hats[0]
plt.plot((0, 10), (beta_hat.dot([1, 0]),
 beta_hat.dot([1, 10])), c='blue',
 label='Initial model')
for beta_hat in beta_hats[1:-1]:
 plt.plot((0, 10),
 (beta_hat.dot([1, 0]),
 beta_hat.dot([1, 10])), c='orange')
beta_hat = beta_hats[-1]
plt.plot((0, 10),
 (beta_hat.dot([1, 0]), beta_hat.dot([1, 10])),
 c='red', label='final model')
plt.gca().set_xlabel('X')
plt.gca().set_ylabel('y')
plt.legend()
plt.show()
```



In [19]: `beta_hats`

Out[19]: `[array([ 0.69106599, -0.6261925 ]),  
array([2.35019342, 1.42954797]),  
array([3.23644661, 1.28608418]),  
array([3.84414902, 1.18771129]),  
array([4.26084958, 1.12025716]),  
array([4.54658049, 1.07400397])]`

In [20]: `# epoch 별 손실을 출력합니다.  
plt.plot(losses)  
plt.gca().set_ylabel('Loss(MSE)')  
plt.gca().set_xlabel('Epoch')  
plt.show()`



```
In [21]: beta_hat_grad, beta_hat
```

```
Out[21]: (array([-0.23565149, 0.0381465]), array([4.54658049, 1.07400397]))
```

## 학습율 (Learning Rate)

경사 하강법의 안정적이고 효율적인 동작을 위해서는 적절한 학습율이 필요합니다.

- 학습율이 너무 크면 모수가 수렴하지 않고 발산합니다.
- 학습율이 너무 작으면 모수가 수렴하기까지 많은 업데이트가 요구됩니다.

**[Ex.2]**

학습율(Learning rate)을 아래와 같이 달리하면서,  
epoch 에 따라 손실(Loss)의 변화를 살펴 봅니다.

```
learning_rates = [0.04, 0.01, 0.001]
```

```
In [22]: '''
위 코드는 서로 다른 학습률을 사용하여
경사 하강법의 성능을 비교하는 시각화를 생성합니다.

1. `plt.subplots(1, 3, figsize=(12, 3))`를 사용하여
 1x3 그리드의 서브플롯을 생성합니다.
 각각의 그래프는 같은 행에 배치되며,
 가로로 총 3개의 그래프가 생성됩니다.
 전체 그림의 크기는 가로 12인치, 세로 3인치입니다.
2. `epochs`는 파라미터 업데이트를 반복할 횟수를
 나타내는 변수입니다. 여기서는 100번의 반복을 수행합니다.
3. `learning_rates`는 서로 다른 학습률의 리스트입니다.
 여기서는 [0.04, 0.01, 0.001]을 사용합니다.
4. `for` 루프를 사용하여 각 학습률에 대해 반복합니다.
 `zip(learning_rates, axes)`를 사용하여
 학습률과 서브플롯을 하나의 쌍으로 묶습니다.
5. 각 학습률에 대해 새로운 리스트 `losses`를 생성합니다.
 이 리스트는 각 반복에서의 손실을 저장합니다.
6. `np.random.uniform(-1, 1, size=2)`를 사용하여
 무작위로 초기화된 파라미터를 생성합니다.
7. `for` 루프를 사용하여 지정된 수의 반복을 수행합니다.
8. 각 반복에서 잔차(residuals)를 계산하고,
 손실(loss)을 `losses` 리스트에 추가합니다.
9. 경사 하강법을 사용하여 모수를 업데이트합니다.
10. 각 학습률에 대한 손실 그래프를 그립니다.
 각 서브플롯에는 해당 학습률에 대한 그래프가 표시됩니다.
11. 서브플롯에 제목(title), x축 레이블(xlabel),
 y축 레이블(ylabel)을 추가합니다.
12. `plt.tight_layout()`를 사용하여
 서브플롯 간의 간격을 조정합니다.
13. `plt.show()`를 호출하여 그림을 출력합니다.
'''
```

이를 통해 서로 다른 학습률을 사용하여  
경사 하강법의 성능을 시각적으로 비교할 수 있습니다.  
'''

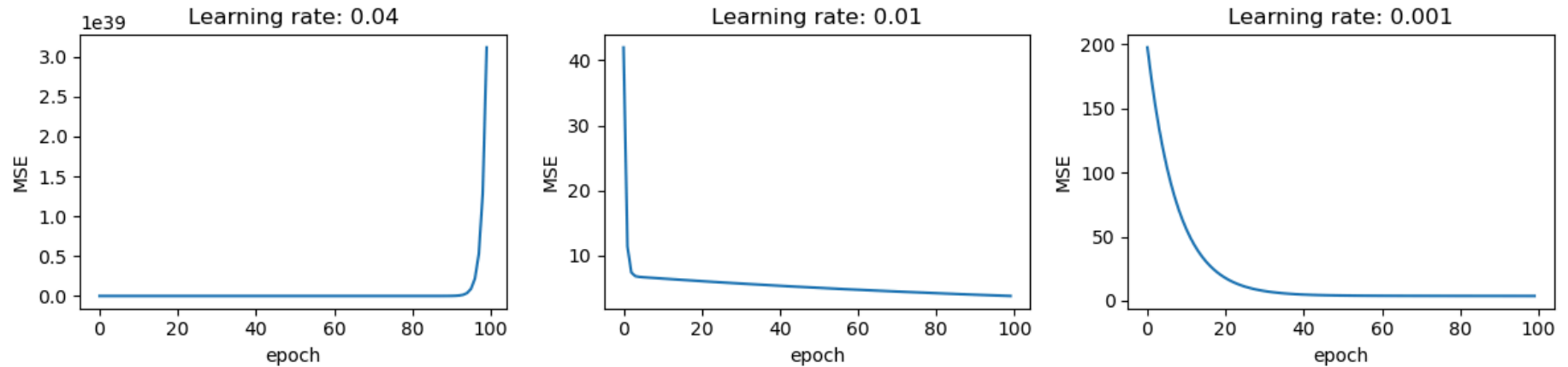
```
fig, axes = plt.subplots(1, 3, figsize=(12, 3))

파라미터 업데이트 루틴의 반복 횟수입니다.
epochs = 100

파라미터 업데이트 비율입니다.
learning_rates = [0.04, 0.01, 0.001]

n = X_add_const.shape[0]

for learning_rate, ax in zip(learning_rates, axes):
 losses = list()
 beta_hat = np.random.uniform(-1, 1, size=2)
 for i in range(epochs):
 residuals = y - np.dot(X_add_const, beta_hat)
 losses.append(np.mean(residuals ** 2))
 beta_hat_grad = -2 * np.dot(X_add_const.T, residuals) / n
 beta_hat -= beta_hat_grad * learning_rate
 ax.plot(losses)
 ax.set_title('Learning rate: {}'.format(learning_rate))
 ax.set_xlabel('epoch')
 ax.set_ylabel('MSE')
plt.tight_layout()
plt.show()
```



## 연속형 변수의 스케일 균일화

다변수 모델에서 변수마다 스케일이 제각각이라면

적합한 변수 별로 학습을 또한 편차가 큽니다.

**경사하강법**이 보다 빠르고 안정적으로 결과를 내려면,

입력 변수 간의 스케일이 고르게 되어 있어야 합니다.

- 연속형 입력 변수들을 표준화나 정규화를 통한 균일화 합니다.

### [Ex.3]

df\_berry 에서 대상 변수를 yield,

입력 변수를 seeds, fruitset, fruitmass로 합니다.

입력 변수의 스케일을 확인하고, 표준화 유무에 따른

경사하강법의 진행 양상을 살펴봅니다.

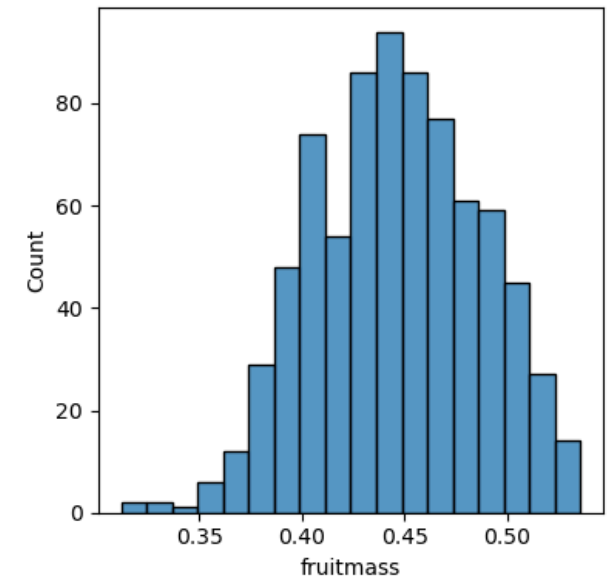
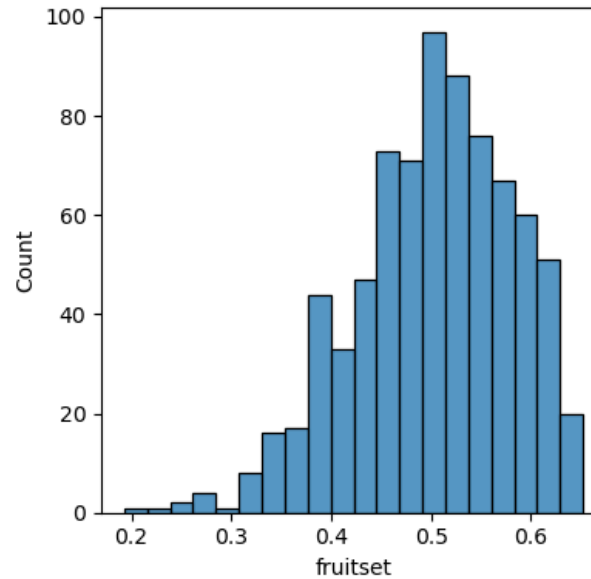
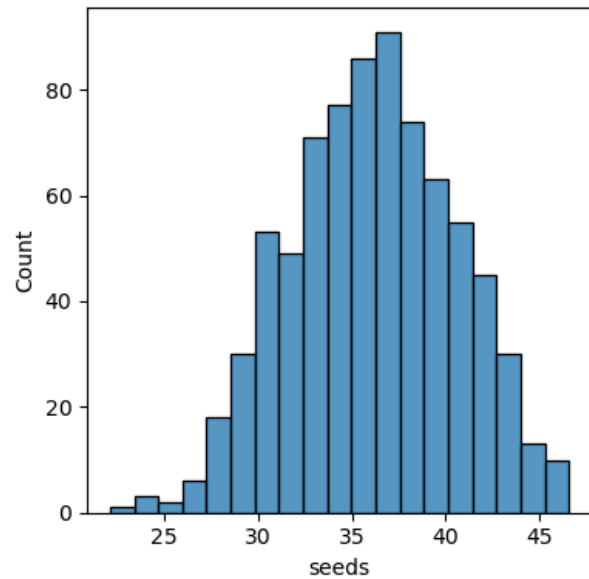
In [23]:

```
'''
코드는 세 가지 변수인 'seeds', 'fruitset', 'fruitmass'의
분포를 히스토그램을 통해 시각화하는 과정을 보여줍니다.

1. `plt.subplots(1, 3, figsize=(12, 4))`를 사용하여
 1x3 그리드의 서브플롯을 생성합니다.
 각각의 그래프는 같은 행에 배치되며,
 가로로 총 3개의 그래프가 생성됩니다.
 전체 그림의 크기는 가로 12인치, 세로 4인치입니다.
2. `X_cols` 리스트에는 'seeds', 'fruitset', 'fruitmass'
 세 가지 변수명이 포함되어 있습니다.
3. `for` 루프를 사용하여 각 변수에 대해 반복합니다.
 `zip(X_cols, axes)`를 사용하여 변수명과 서브플롯을
 하나의 쌍으로 묶습니다.
4. `sns.histplot(data=df_berry, x=i, ax=ax)`를 사용하여
 seaborn 라이브러리를 활용하여 데이터프레임 `df_berry`에서
 변수 `i`에 대한 히스토그램을 그립니다.
 `ax=ax`를 통해 각 히스토그램을 해당하는
 서브플롯에 그립니다.
5. `plt.tight_layout()`를 사용하여
 서브플롯 간의 간격을 조정합니다.

이를 통해 'seeds', 'fruitset', 'fruitmass' 변수의
분포를 히스토그램으로 시각화하여 확인할 수 있습니다.
'''

히스토그램을 이용하여 seeds, fruitset, fruitmass의 분포 확인
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
X_cols = ['seeds', 'fruitset', 'fruitmass']
for i, ax in zip(X_cols, axes):
 sns.histplot(data=df_berry, x=i, ax=ax)
plt.tight_layout()
plt.show()
```



In [24]:

'''

코드는 다음과 같은 작업을 수행합니다:

1. ``df_berry`` 데이터프레임에서  
    `'seeds', 'fruitset', 'fruitmass'` 열을 선택합니다.
2. ``assign(const=1)``을 사용하여 새로운  
    `'const'` 열을 추가합니다.  
    이 열은 상수항을 나타내며, 모든 값이 1로 채워집니다.
3. ``.values``를 사용하여 데이터프레임을  
    넘파이 배열로 변환합니다.
4. 마지막으로, 처음 5개의 행을 출력합니다.

결과적으로는 다음과 같은 형태의 배열이 생성됩니다:

```
array([[x1_1, x2_1, x3_1, 1],
 [x1_2, x2_2, x3_2, 1],
 [x1_3, x2_3, x3_3, 1],
 [x1_4, x2_4, x3_4, 1],
 [x1_5, x2_5, x3_5, 1]])
```

여기서 각 행은 하나의 데이터 포인트를 나타내며,  
`'seeds', 'fruitset', 'fruitmass'` 열의 값과

상수항 (1)으로 이루어져 있습니다.  
이렇게 구성된 배열은 선형 회귀 모델에서  
설명 변수와 함께 사용됩니다.  
'''

```
별도로 절편을 계산하지 않고,
상수항을 포함시켜 절편을 계산합니다.
X_berry_const = df_berry[X_cols].assign(
 const = 1
).values
X_berry_const[:5]
```

```
Out[24]: array([[31.67889844, 0.41065206, 0.40815901, 1.],
 [33.44938499, 0.44425436, 0.42545764, 1.],
 [30.54630597, 0.38378651, 0.3991724 , 1.],
 [31.56258602, 0.40756415, 0.40878935, 1.],
 [28.87371362, 0.35441305, 0.38270285, 1.]])
```

```
In [25]: '''
코드는 세 가지 다른 학습률을 사용하여
선형 회귀 모델의 성능을 비교하는 그래프를 생성합니다.
```

1. `plt.subplots(1, 3, figsize=(12, 3))`를 사용하여  
1x3 그리드의 서브플롯을 생성합니다.  
각각의 그래프는 같은 행에 배치되며,  
가로로 총 3개의 그래프가 생성됩니다.  
전체 그림의 크기는 가로 12인치, 세로 3인치입니다.
2. `epochs` 변수에는 파라미터 업데이트를 반복할 횟수를  
나타내는 값을 지정합니다. 여기서는 50번 반복을 수행합니다.
3. `learning_rate` 변수에는 경사 하강법에서 사용할  
학습률을 설정합니다. 초기에는 `1e-5`로 설정되어 있습니다.
4. `learning_rates` 리스트에는 서로 다른 학습률의 값이  
포함되어 있습니다.  
이 값들은 `[1e-3, 1e-4, 1e-5]`로 설정되어 있습니다.
5. `for` 루프를 사용하여 각 학습률에 대해 반복합니다.  
`zip(learning_rates, axes)`를 사용하여  
학습률과 서브플롯을 하나의 쌍으로 묶습니다.
6. 각 학습률에 대해 새로운 리스트 `losses`를 생성합니다.  
이 리스트는 각 반복에서의 손실을 저장합니다.
7. `np.random.uniform(-1, 1, size=len(X_cols) + 1)`를  
사용하여 무작위로 초기화된 초기 파라미터를 생성합니다.

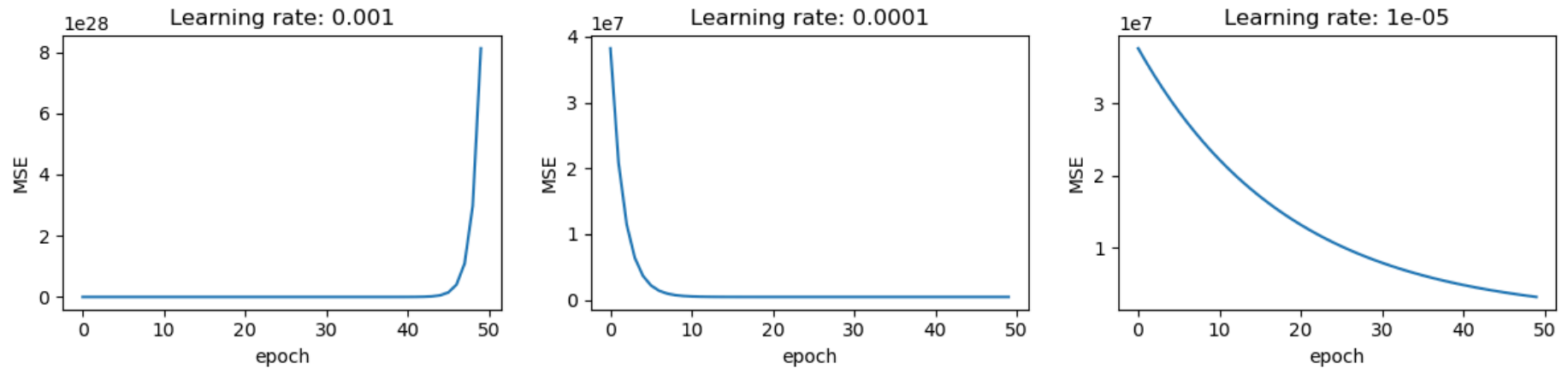
- 이때, `len(X_cols) + 1`은 설명 변수의 수와 상수항을 포함하여 파라미터의 총 개수를 나타냅니다.
8. `for` 루프를 사용하여 지정된 수의 반복을 수행합니다.
  9. 각 반복에서 잔차(residuals)를 계산하고, 손실(loss)을 `losses` 리스트에 추가합니다.
  10. 경사 하강법을 사용하여 모수를 업데이트합니다.
  11. 각 학습률에 대한 손실 그래프를 그립니다.  
각 서브플롯에는 해당하는 학습률에 대한 그래프가 표시됩니다.
  12. 서브플롯에 제목(title), x축 레이블(xlabel), y축 레이블(ylabel)을 추가합니다.
  13. `plt.tight_layout()`를 사용하여 서브플롯 간의 간격을 조정합니다.

이를 통해 서로 다른 학습률을 사용하여 경사 하강법의 성능을 시각적으로 비교할 수 있습니다.

```
'''
```

```
fig, axes = plt.subplots(1, 3, figsize=(12, 3))
epochs = 50
learning_rate = 1e-5 # 파라미터 업데이트 비율입니다.
n = X_berry_const.shape[0]

learning_rates = [1e-3, 1e-4, 1e-5]
for learning_rate, ax in zip(learning_rates, axes):
 losses = list()
 beta_hat = np.random.uniform(-1, 1, size=len(X_cols) + 1)
 for i in range(epochs):
 residuals = df_berry['yield'] - np.dot(X_berry_const, beta_hat)
 losses.append(np.mean(residuals ** 2))
 beta_hat_grad = -2 * np.dot(X_berry_const.T, residuals) / n
 beta_hat -= beta_hat_grad * learning_rate
 ax.plot(losses)
 ax.set_title('Learning rate: {}'.format(learning_rate))
 ax.set_xlabel('epoch')
 ax.set_ylabel('MSE')
plt.tight_layout()
plt.show()
```



In [26]:

```
'''
코드는 다음과 같은 작업을 수행합니다:

1. `df_berry[X_cols]`를 사용하여
 'seeds', 'fruitset', 'fruitmass' 열을 선택합니다.
2. `apply()` 함수를 사용하여 각 열에 대해 표준화를 수행합니다.
 표준화는 각 값에서 열의 평균을 빼고
 그 결과를 열의 표준편차로 나누어 줌으로써 이루어집니다.
3. 람다 함수(lambda function) `(x - x.mean()) / x.std()`는
 각 열에 대해 평균을 빼고 표준편차로 나누어
 표준화를 수행합니다.
4. `assign(const=1)`을 사용하여 상수항을 추가합니다.
 이 상수항은 모든 값이 1로 이루어진 열로,
 회귀 모델의 절편을 나타냅니다.
5. `.values`를 사용하여 데이터프레임을
 넘파이 배열로 변환합니다.
6. 처음 5개의 행을 출력합니다.

결과적으로 생성된 배열은 다음과 같은 형태를 가집니다:

array([[standardized_x1_1, standardized_x2_1, standardized_x3_1, 1],
 [standardized_x1_2, standardized_x2_2, standardized_x3_2, 1],
 [standardized_x1_3, standardized_x2_3, standardized_x3_3, 1],
 [standardized_x1_4, standardized_x2_4, standardized_x3_4, 1],
 [standardized_x1_5, standardized_x2_5, standardized_x3_5, 1]])
'''
```

```
'''
여기서 각 행은 하나의 데이터 포인트를 나타내며,
'seeds', 'fruitset', 'fruitmass' 열의 값이 표준화되고
상수항(1)이 추가되어 있습니다.
이렇게 구성된 배열은 선형 회귀 모델에 사용됩니다.
'''
```

```
입력 변수를 표준화 시킵니다.
X_berry_std_const \
 = df_berry[X_cols].apply(\
 lambda x: (x - x.mean()) / x.std()) \
 .assign(const = 1).values
X_berry_std_const[:5]
```

```
Out[26]: array([[-1.01499449, -1.1513429 , -0.93780916, 1.],
 [-0.61057889, -0.72838037, -0.50891035, 1.],
 [-1.27370194, -1.48950797, -1.16062122, 1.],
 [-1.04156264, -1.19021135, -0.92218059, 1.],
 [-1.65575648, -1.85924081, -1.56896392, 1.]])
```

```
In [16]: '''
코드는 다음과 같은 작업을 수행합니다:
```

1. `plt.subplots(1, 3, figsize=(12, 3))`를 사용하여 1x3 그리드의 서브플롯을 생성합니다. 각 그래프는 동일한 행에 위치하며, 총 3개의 그래프가 생성됩니다. 전체 그림의 크기는 가로 12인치, 세로 3인치로 설정됩니다.
2. `epochs` 변수에는 파라미터 업데이트를 반복할 횟수를 설정합니다. 여기서는 50회로 설정됩니다.
3. `learning_rate` 변수는 경사 하강법에서 사용할 학습률을 설정합니다. 초기에는 `1e-5`로 설정됩니다.
4. `learning_rates` 리스트에는 서로 다른 학습률의 값들이 포함됩니다. 여기서는 `[1e-1, 1e-2, 1e-3]`로 설정되어 있습니다.
5. `for` 루프를 사용하여 각 학습률에 대해 반복합니다. `zip(learning_rates, axes)`를 사용하여 학습률과 해당하는 서브플롯을 묶습니다.
6. 각 학습률에 대해 새로운 리스트 `losses`를 생성합니다. 이 리스트는 각 반복에서의 손실을 저장합니다.
7. `np.random.uniform(-1, 1, size=len(X_cols) + 1)`를 사용하여 무작위로 초기화된 초기 파라미터를 생성합니다. 여기서 `len(X_cols) + 1`은 설명 변수의 수와

- 상수항을 포함하여 파라미터의 총 개수를 나타냅니다.
8. ``for`` 루프를 사용하여 지정된 수의 반복을 수행합니다.
  9. 각 반복에서 잔차(residuals)를 계산하고, 손실(loss)을 ``losses`` 리스트에 추가합니다.
  10. 경사 하강법을 사용하여 모수를 업데이트합니다.
  11. 각 학습률에 대한 손실 그래프를 그립니다.  
각 서브플롯에는 해당하는 학습률에 대한 그래프가 표시됩니다.
  12. 서브플롯에 제목(title), x축 레이블(xlabel), y축 레이블(ylabel)을 추가합니다.
  13. ``plt.tight_layout()``을 사용하여 서브플롯 간의 간격을 조정합니다.

이를 통해 각 학습률에 따른 모델의 학습 과정을 시각적으로 비교할 수 있습니다.

학습률이 높은 경우(0.1) 손실이 빠르게 감소하지만, 일정 수준 이상에서는 수렴하지 않고 발산하는 경향이 있습니다. 학습률이 낮은 경우(0.001) 손실이 천천히 감소하지만, 더 안정적으로 수렴하는 것을 볼 수 있습니다. 중간 학습률(0.01)은 적당한 수준의 손실 감소와 안정적인 수렴을 보여줍니다. 이 그래프를 통해 적절한 학습률을 선택할 수 있습니다.

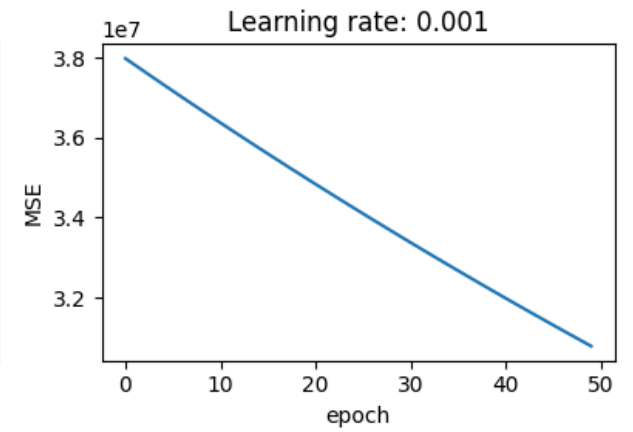
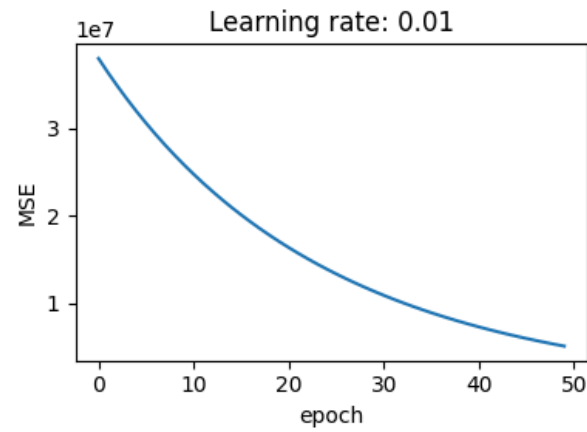
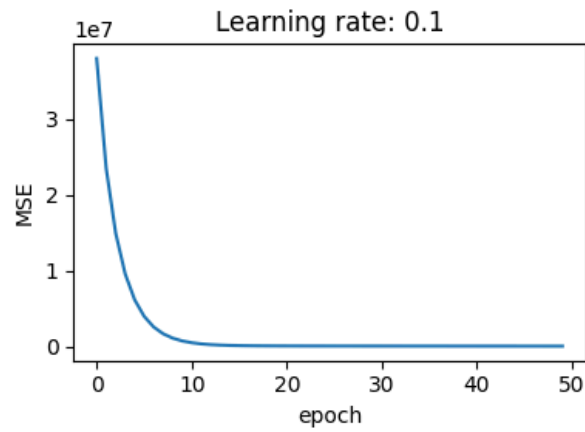
```
fig, axes = plt.subplots(1, 3, figsize=(12, 3))
epochs = 50
learning_rate = 1e-5 # 파라미터 업데이트 비율입니다.
n = X_berry_const.shape[0]

learning_rates = [1e-1, 1e-2, 1e-3]
for learning_rate, ax in zip(learning_rates, axes):
 losses = list()
 beta_hat = np.random.uniform(-1, 1,
 size=len(X_cols) + 1)

 for i in range(epochs):
 residuals
 = df_berry['yield'] - np.dot(X_berry_std_const,
 beta_hat)

 losses.append(np.mean(residuals ** 2))
 beta_hat_grad
 = -2 * np.dot(X_berry_std_const.T, residuals) / n
```

```
beta_hat -= beta_hat_grad * learning_rate
ax.plot(losses)
ax.set_title('Learning rate: {}'.format(learning_rate))
ax.set_xlabel('epoch')
ax.set_ylabel('MSE')
plt.tight_layout()
plt.show()
```



In [27]: Image('135.png')

Out [27]: **확률적 경사 하강법(Stochastic Gradient Descent)**

학습 데이터가 고르게 분포한다면, 일부분 데이터의 경사도와 전체 데이터의 경사도가 유사합니다.

$$\frac{dL_{batch}}{d\hat{\beta}} \approx \frac{dL}{d\hat{\beta}}$$

1 epoch에서 전체 데이터셋을 일부분씩 나누어 경사를 구하고 업데이트하여, 경사도 계산 시간과 메모리를 줄여 효율적으로 경사하강법을 수행합니다.

- 전체에서 경사(Gradient)를 구할 때의 결과는 늘 같지만, 나누어서 경사를 구할 때는 어떻게 썰이 나뉘느냐에 따라서 결과가 달라지게 되어 확률적 (Stochastic) 경사 하강법이라고 합니다.
- 배치(Batch): 전체의 데이터셋을 작은 단위로 나뉘어진 데이터셋들을 말합니다

이 배치(Batch)의 크기가 1인 경우를 Stochastic Gradient Descent라고 1 보다 클 경우를 (Batch Gradient Descent)라고 구분하기도 합니다.

- 일부분의 데이터를 가지고 학습이 이루어지므로 굳이 모든 데이터를 메모리에 올릴 필요없이 배치를 만드는데 필요한 만큼만 메모리에 올리면 되므로 메모리 효율적이고, 대용량의 데이터를 학습할 수 있습니다.
- 일부의 데이터로 파라미터의 업데이트를 하므로 수렴이 상대적으로 적은 수의 반복으로 빠르게 될 가능성이 높습니다.
- 데이터의 분포가 고르지 않고 편중되어 있다면 제대로 학습이 되기 어렵습니다. 편중되지 않도록 데이터를 잘 섞어주어야 합니다.

**[Ex.4]**

[Ex. 3]에서 배치(batch)의 크기를 32로 하여 배치(Batch) 경사하강법으로 학습을 합니다.

In [17]:

```
'''
코드는 미니 배치 경사 하강법을 사용하여
선형 회귀 모델을 학습하는 과정을 나타냅니다.

1. `beta_hat = np.random.uniform(-1, 1, size=len(X_cols) + 1)`:
 초기 파라미터를 무작위로 설정합니다.
 이때 `len(X_cols) + 1`은 설명 변수의 수와 상수항을
 포함하여 파라미터의 총 개수를 나타냅니다.
2. `epochs = 50`:
 전체 데이터셋에 대해 반복할 횟수를 설정합니다.
3. `batch_size = 32`:
 미니 배치의 크기를 설정합니다.
 전체 데이터를 작은 배치로 나누어
```

각 반복에서 사용됩니다.

4. ``learning_rate = 5e-2``: 학습률을 설정합니다.  
파라미터 업데이트의 속도를 결정하는  
하이퍼파라미터입니다.
5. ``n = len(df_berry)``:  
전체 데이터셋의 크기를 저장합니다.
6. ``losses = list()``:  
손실(평균 제곱 오차)를 저장할 빈 리스트를 생성합니다.
7. ``for i in range(epochs):``:  
주어진 epoch 횟수 동안 반복합니다.
8. ``for j in range(0, n, batch_size):``:  
미니 배치를 사용하여 데이터를 반복적으로 처리합니다.  
전체 데이터셋을 배치로 나누어 처리합니다.
9. ``X_batch = X_berry_std_const[j:j+batch_size]``:  
현재 배치에 해당하는 입력 데이터를 선택합니다.
10. ``residuals = df_berry.iloc[j:j+batch_size]['yield']``  
- ``np.dot(X_batch, beta_hat)``:  
현재 배치에 대한 예측값과 실제값의 차이(잔차)를 계산합니다.
11. ``beta_hat_grad = -2 * np.dot(X_batch.T, residuals)``  
- ``/ X_batch.shape[0]``:  
현재 배치에 대한 파라미터의  
그래디언트(기울기)를 계산합니다.
12. ``beta_hat -= beta_hat_grad * learning_rate``:  
그래디언트를 사용하여 파라미터를 업데이트합니다.  
경사 하강법의 공식을 적용하여 파라미터를 조정합니다.
13. ``losses.append(np.mean((df_berry['yield']`  
- `np.dot(X_berry_std_const, beta_hat)) ** 2))`:  
현재 epoch에서 전체 데이터셋에 대한  
평균 제곱 오차를 계산하고, 손실 리스트에 추가합니다.`
14. ``plt.plot(losses)``: 손실 그래프를 그립니다.  
각 epoch에서 손실이 어떻게 감소하는지를  
시각적으로 확인할 수 있습니다.
15. ``plt.gca().set_ylabel('Loss(MSE))``:  
y축 레이블을 설정합니다.  
손실 함수의 값이 표시됩니다.
16. ``plt.gca().set_xlabel('Epoch')``:  
x축 레이블을 설정합니다.  
반복(epoch)의 횟수가 표시됩니다.

이를 통해 미니 배치 경사 하강법을 통해  
선형 회귀 모델을 학습하고,

학습 과정에서 손실이 어떻게 감소하는지를 시각적으로 확인할 수 있습니다. 미니 배치 경사 하강법은 전체 데이터셋을 한 번에 처리하는 것이 아니라 작은 배치 단위로 나누어 처리하여 계산량을 줄이고 학습 속도를 향상시키는 방법입니다. 이를 통해 전체 데이터셋이 너무 클 때도 효율적으로 학습할 수 있습니다. 이것은 확률적 경사 하강법(Stochastic Gradient Descent, SGD)과 더 큰 배치를 사용하는 배치 경사 하강법 사이의 절충안으로 볼 수 있습니다. 미니 배치의 크기를 조정함으로써 계산 효율성과 학습의 안정성 사이의 균형을 조정할 수 있습니다.

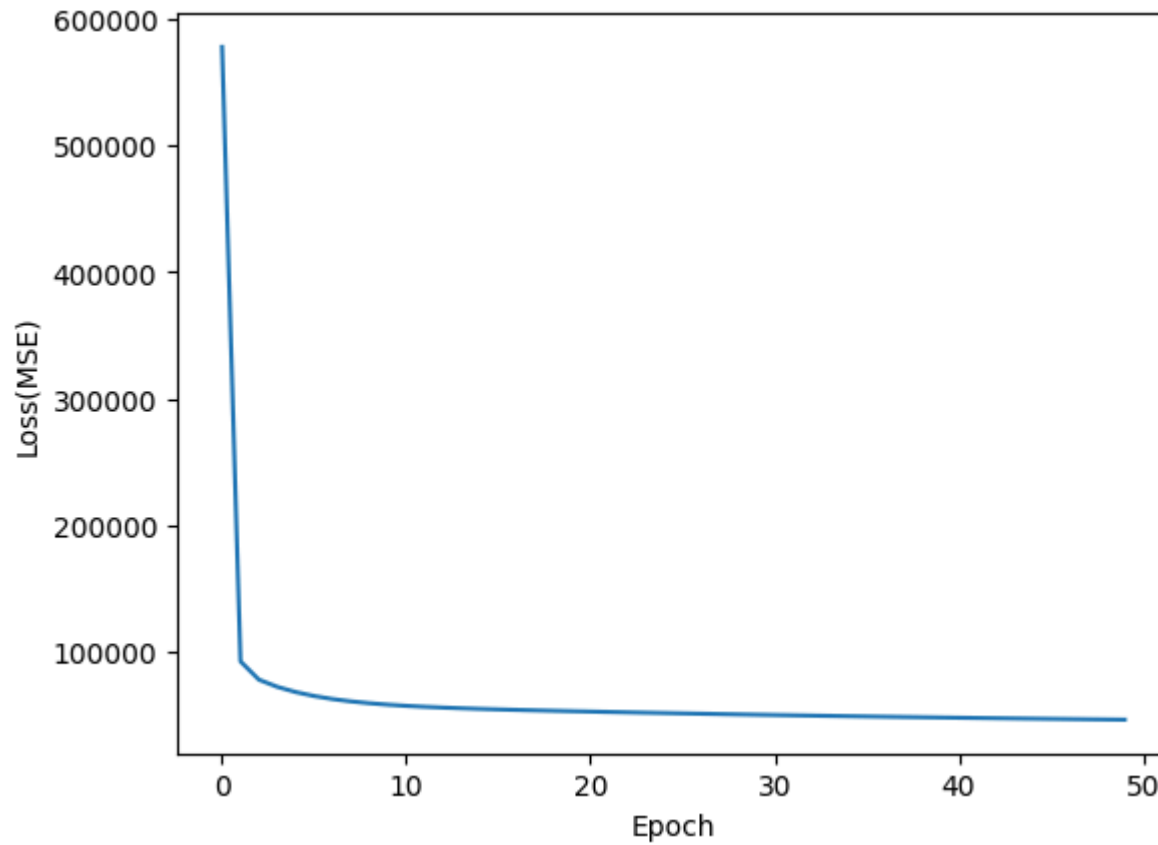
```
beta_hat = np.random.uniform(-1, 1, size=len(X_cols) + 1)
beta_hat

epochs = 50
batch_size = 32

파라미터 업데이트 비율입니다.
learning_rate = 5e-2
n = len(df_berry)
losses = list()
for i in range(epochs):
 for j in range(0, n, batch_size):
 X_batch = X_berry_std_const[j:j+batch_size]
 residuals = df_berry.iloc[j:j+batch_size]['yield'] \
 - np.dot(X_batch, beta_hat)
 beta_hat_grad \
 = -2 * np.dot(X_batch.T, residuals) / X_batch.shape[0]
 beta_hat -= beta_hat_grad * learning_rate

 losses.append(np.mean((df_berry['yield'] \
 - np.dot(X_berry_std_const, beta_hat)) ** 2))
```

```
In [18]: plt.plot(losses)
plt.gca().set_ylabel('Loss(MSE)')
plt.gca().set_xlabel('Epoch')
plt.show()
```



### 조기 종료(Early Stopping)

학습셋의 일부를 분리하여 매 Epoch 마다 성능을 측정하여,

이 때 성능이 개선이 없으면 학습을 종료하도록 합니다.

이를 통해 불필요한 학습 과정을 생략하고,

과도하게 학습 데이터에 맞춰지지 않도록 합니다.

**[Ex.5]**

[Ex. 4]에서 학습 데이터의 10% 만큼을

조기 종료 데이터로 만들어 조기 종료 루틴이 포함된

경사 하강법을 수행합니다.

In [19]:

```
'''
코드는 조기 종료를 사용하여 모델의 과적합을 방지하고
최적의 학습 횟수를 결정하는 과정을 나타냅니다.

1. `beta_hat = np.random.uniform(-1, 1, size=len(X_cols) + 1)`:
 초기 파라미터를 무작위로 설정합니다.
 이때 `len(X_cols) + 1`은 설명 변수의 수와
 상수항을 포함하여 파라미터의 총 개수를 나타냅니다.
2. `epochs = 1000`:
 전체 데이터셋에 대해 반복할 횟수를 설정합니다.
3. `batch_size = 32`:
 미니 배치의 크기를 설정합니다. 전체 데이터를
 작은 배치로 나누어 각 반복에서 사용됩니다.
4. `learning_rate = 1e-2`:
 학습률을 설정합니다. 파라미터 업데이트의 속도를
 결정하는 하이퍼 파라미터입니다.
5. `X_berry_std_fit, X_berry_std_es, y_berry_fit,
 y_berry_es = train_test_split(...)`:
 전체 데이터셋을 학습 세트와 검증 세트로 나눕니다.
6. `n = X_berry_std_fit.shape[0]`:
 학습 세트의 샘플 개수를 저장합니다.
7. `prev_loss = np.inf`:
 이전 검증 세트의 손실을 무한대로 설정합니다.
8. `for i in range(epochs):`:
 주어진 epoch 횟수 동안 반복합니다.
9. `for j in range(0, n, batch_size):`:
 미니 배치를 사용하여 데이터를 반복적으로 처리합니다.
 전체 데이터셋을 배치로 나누어 처리합니다.
10. `es_loss = np.mean((y_berry_es
 - np.dot(X_berry_std_es, beta_hat)) ** 2)`:
 검증 세트에 대한 손실을 계산합니다.
```

```

11. `if prev_loss < es_loss: ...`:
 이전 검증 세트의 손실이 현재 손실보다 작은 경우,
 과적합이 시작되었음을 의미하므로
 조기 종료를 수행합니다.
 이때 `prev_loss`는 이전 검증 세트의 손실을 나타내며,
 `es_loss`는 현재 검증 세트의 손실을 나타냅니다.
12. `prev_loss = es_loss`:
 이전 검증 세트의 손실을 현재 손실로 업데이트합니다.
13. `losses.append(np.mean((y_berry_fit
 - np.dot(X_berry_std_fit, beta_hat)) ** 2))`:
 학습 세트에 대한 손실을 계산하고, 손실 리스트에 추가합니다.
14. `plt.plot(losses)`: 손실 그래프를 그립니다.
 각 epoch에서 손실이 어떻게 감소하는지를
 시각적으로 확인할 수 있습니다.
15. `plt.gca().set_ylabel('Loss(MSE))`:
 y축 레이블을 설정합니다. 손실 함수의 값이 표시됩니다.
16. `plt.gca().set_xlabel('Epoch')`:
 x축 레이블을 설정합니다. 반복(epoch)의 횟수가 표시됩니다.

```

이를 통해 검증 세트의 손실이 더 이상 감소하지 않을 때  
 학습을 조기에 중단하여 과적합을 방지할 수 있습니다.  
 ...

```

from sklearn.model_selection import train_test_split
beta_hat = np.random.uniform(-1, 1,
 size=len(X_cols) + 1)

beta_hat

epochs = 1000
batch_size = 32

파라미터 업데이트 비율입니다.
learning_rate = 1e-2
X_berry_std_fit, X_berry_std_es, y_berry_fit, y_berry_es
 = train_test_split(X_berry_std_const,
 df_berry['yield'],
 train_size=0.9)
n = X_berry_std_fit.shape[0]
losses = list()
prev_loss = np.inf

```

```

for i in range(epochs):
 for j in range(0, n, batch_size):
 X_batch = X_berry_std_fit[j:j+batch_size]

 residuals \
 = y_berry_fit[j:j+batch_size] \
 - np.dot(X_batch, beta_hat)

 beta_hat_grad \
 = -2 * np.dot(X_batch.T, residuals) / X_batch.shape[0]

 beta_hat \
 -= beta_hat_grad * learning_rate

 es_loss \
 = np.mean((y_berry_es \
 - np.dot(X_berry_std_es, beta_hat)) ** 2)

 if prev_loss < es_loss:
 print('Early Stopped: {}, {}, {}'.format(i + 1, prev_loss, es_loss))
 break

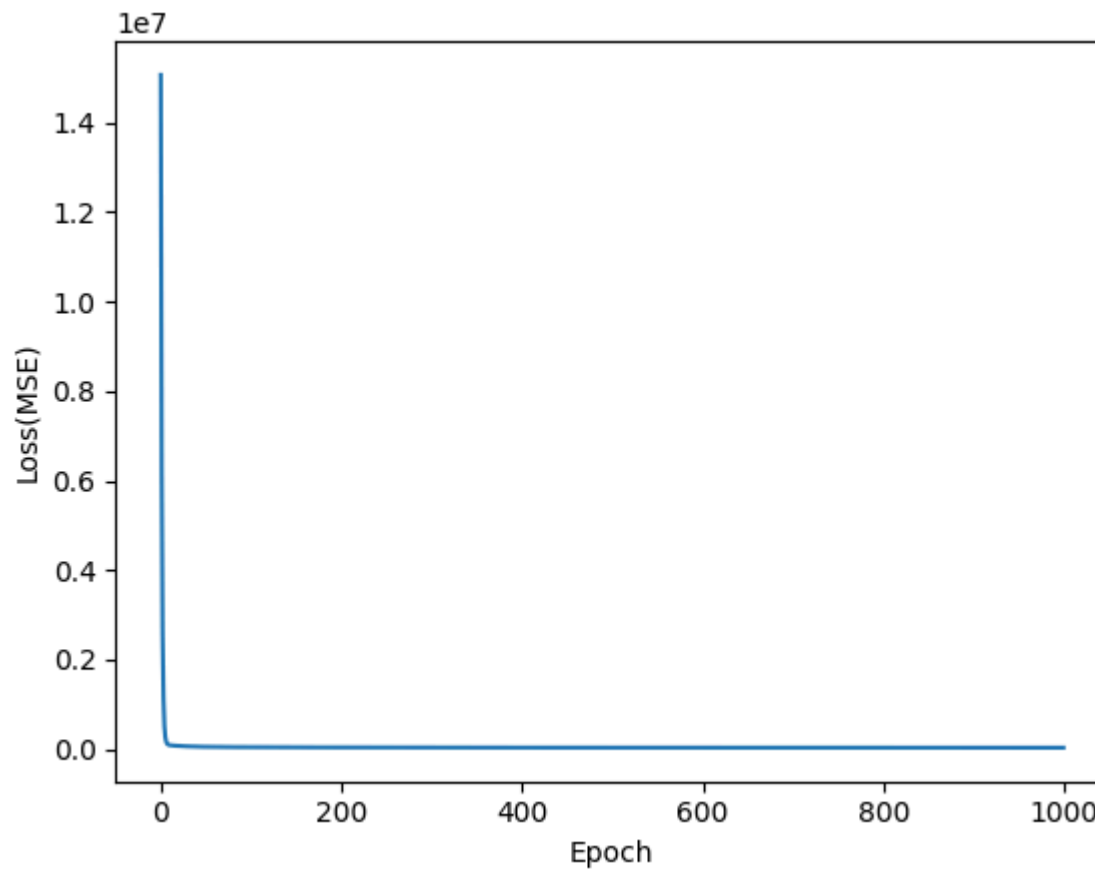
 prev_loss = es_loss
 losses.append(np.mean((y_berry_fit \
 - np.dot(X_berry_std_fit, beta_hat)) ** 2))

```

```

In [20]: plt.plot(losses)
plt.gca().set_ylabel('Loss(MSE)')
plt.gca().set_xlabel('Epoch')
plt.show()

```



```
In [28]: Image('136.png')
```

Out[28]: **학습률 스케줄링**

학습률을 진행상황에 따라 조정합니다.

학습이 진행이 될 수록 학습률을 줄여

보다 안정적으로 파라미터를 수렴시키도록 하고,

또한 보다 세밀하게 파라미터의 최적공간을 탐색하도록합니다.

지수 감소(Exponential Decay):  $\text{learning\_rate}(\text{학습률}) = \text{Initial learning rate}(\text{초기 학습률}) \times \text{감소율}^t$

$t$ : epoch 수

[Ex.6]

[Ex. 4]에서 학습률 스케줄링을 추가하고,

학습률 감소율을 0.98로 설정하여 학습을 해봅니다.

In [21]:

```
'''
코드는 learning rate를 epoch에 따라 점차적으로
감소시키는 방법을 사용하여 학습을 수행합니다.
이것은 모델이 수렴하는 방향으로 학습률을 조정하여
최적의 솔루션에 더 가깝게 만듭니다.

1. `beta_hat = np.random.uniform(-1, 1, size=len(X_cols) + 1)`:
 초기 파라미터를 무작위로 설정합니다.
 이때 `len(X_cols) + 1`은 설명 변수의 수와 상수항을
 포함하여 파라미터의 총 개수를 나타냅니다.
2. `epochs = 50`:
 전체 데이터셋에 대해 반복할 횟수를 설정합니다.
3. `batch_size = 32`:
 미니 배치의 크기를 설정합니다. 전체 데이터를
 작은 배치로 나누어 각 반복에서 사용됩니다.
'''
```

```

4. `learning_rate_init = 1e-2`:
 초기 학습률을 설정합니다. 파라미터 업데이트의
 속도를 결정하는 하이퍼 파라미터입니다.
5. `rate = 0.95`: 학습률 감소율을 설정합니다.
 각 epoch 마다 학습률이 이전 학습률에 곱해져서 감소됩니다.
6. `X_berry_std_fit, X_berry_std_es, y_berry_fit,
 y_berry_es = train_test_split(...)` :
 전체 데이터셋을 학습 세트와 검증 세트로 나눕니다.
7. `n = X_berry_std_fit.shape[0]`:
 학습 세트의 샘플 개수를 저장합니다.
8. `prev_loss = np.inf`:
 이전 검증 세트의 손실을 무한대로 설정합니다.
9. `for i in range(epochs):`:
 주어진 epoch 횟수 동안 반복합니다.
10. `learning_rate = learning_rate_init * rate ** i`:
 현재 epoch에 해당하는 학습률을 계산합니다.
 이것은 초기 학습률을 학습률 감소율의 제곱값을
 현재 epoch 수로 나눈 값입니다.
11. `for j in range(0, n, batch_size):`:
 미니 배치를 사용하여 데이터를 반복적으로 처리합니다.
 전체 데이터셋을 배치로 나누어 처리합니다.
12. `es_loss = np.mean((y_berry_es
 - np.dot(X_berry_std_es, beta_hat)) ** 2)`:
 검증 세트에 대한 손실을 계산합니다.
13. `if prev_loss < es_loss: ...`:
 이전 검증 세트의 손실이 현재 손실보다 작은 경우,
 과적합이 시작되었음을 의미하므로
 조기 종료를 수행합니다.
 이때 `prev_loss`는 이전 검증 세트의 손실을 나타내며,
 `es_loss`는 현재 검증 세트의 손실을 나타냅니다.
14. `prev_loss = es_loss`:
 이전 검증 세트의 손실을 현재 손실로 업데이트합니다.
15. `losses.append(np.mean((y_berry_fit
 - np.dot(X_berry_std_fit, beta_hat)) ** 2))`:
 학습 세트에 대한 손실을 계산하고,
 손실 리스트에 추가합니다.
16. `plt.plot(losses):` : 손실 그래프를 그립니다.
 각 epoch에서 손실이 어떻게 감소하는지를
 시각적으로 확인할 수 있습니다.
17. `plt.gca().set_ylabel('Loss(MSE)')`:
 y축 레이블을 설정합니다. 손실 함수의 값이 표시됩니다.

```

```

18. `plt.gca().set_xlabel('Epoch')`:
 x축 레이블을 설정합니다. 반복(epoch) 횟수가 표시됩니다.

이렇게 함으로써 학습률이 점차적으로 감소함에 따라
모델의 성능을 개선할 수 있습니다.
'''

from sklearn.model_selection import train_test_split

beta_hat = np.random.uniform(-1, 1, size=len(X_cols) + 1)
beta_hat

epochs = 50
batch_size = 32

파라미터 업데이트 비율입니다.
learning_rate_init = 1e-2
rate = 0.95
X_berry_std_fit, X_berry_std_es, y_berry_fit, y_berry_es
 = train_test_split(X_berry_std_const, df_berry['yield'],
 train_size=0.9)

n = X_berry_std_fit.shape[0]
losses = list()
learning_rates = list()
prev_loss = np.inf

for i in range(epochs):
 learning_rate = learning_rate_init * rate ** i
 learning_rates.append(learning_rate)
 for j in range(0, n, batch_size):
 X_batch = X_berry_std_fit[j:j+batch_size]
 residuals \
 = y_berry_fit[j:j+batch_size] - np.dot(X_batch, beta_hat)
 beta_hat_grad \
 = -2 * np.dot(X_batch.T, residuals) / X_batch.shape[0]
 beta_hat \
 -= beta_hat_grad * learning_rate
 es_loss \
 = np.mean((y_berry_es - np.dot(X_berry_std_es, beta_hat)) ** 2)

```

```

if prev_loss < es_loss:
 print('Early Stopped: {}, {}, {}'.format(i + 1, prev_loss, es_loss))
 break

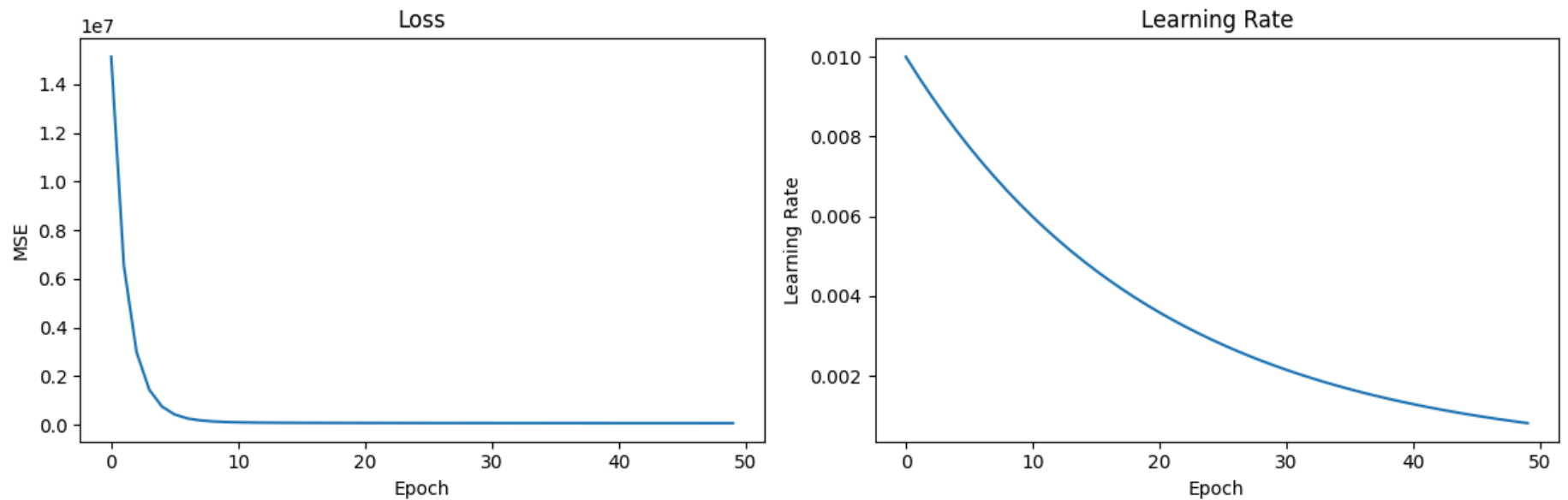
prev_loss = es_loss
losses.append(np.mean((y_berry_fit \
 - np.dot(X_berry_std_fit, beta_hat)) ** 2))

```

```

In [22]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(losses)
axes[0].set_ylabel('MSE')
axes[0].set_xlabel('Epoch')
axes[0].set_title("Loss")
axes[1].plot(learning_rates)
axes[1].set_ylabel('Learning Rate')
axes[1].set_xlabel('Epoch')
axes[1].set_title("Learning Rate")
plt.tight_layout()
plt.show()

```



In [29]: `Image('137.png')`

Out[29]: **2. EM 알고리즘(Expectation Maximization)**

EM 알고리즘은 미지의 변수(latent variables)를 포함하고 있는 통계 모델의 모수를 구하는 알고리즘입니다.

### 상세 배경

1. 확률 모델은  $X$ : 관측 데이터,  $Z$ : 미지의 변수,  $\theta$ : 모수 벡터와 우도 함수  $L(\theta; X, Z) = p(X, Z|\theta)$  로 구성됩니다.
2. 최대우도추정을 통한 모수의 추정 은 관측데이터의 주변 우도를 최대화하는 수를 모수로 추정하는 것입니다.
3. 2를 공식화하면 아래와 같습니다.

$$\theta = \operatorname{argmax}_{\theta} L(\theta; X)$$

$$L(\theta; X) = p(X|\theta) = \int p(X, Z|\theta) dZ = \int p(X|Z, \theta) p(Z|\theta) dZ$$

하지만,  $Z$ 는 미지의 변수이기 때문에,  $\theta$ 를 알기전에는  $Z$ 의 분포를 알 수 없습니다.

아래의 코드는 두 개의 정규 분포에서 생성한 표본입니다.

$N(-10, 9)$  에서 300개의 표본,  $N(10, 4)$ 에서 550개

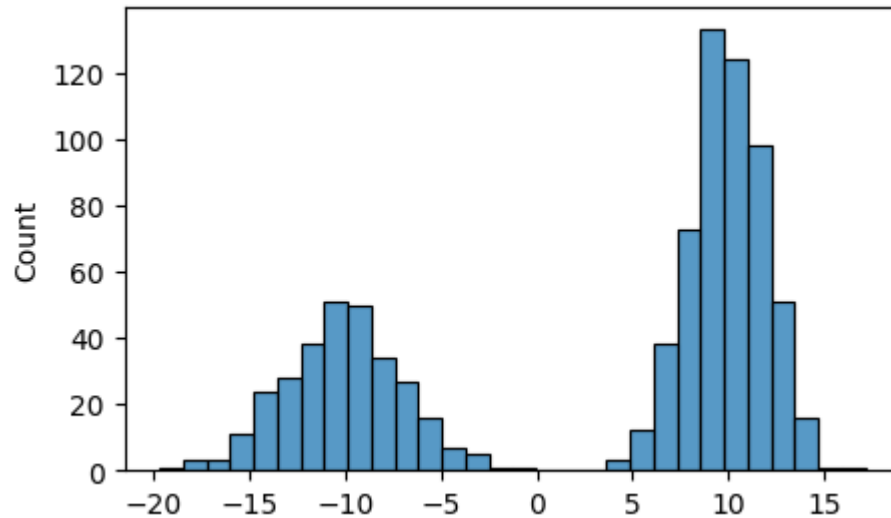
이 표본들의 모수를 모르고 또한 표본들이 속한 모집단을 모르는 상태에서

두 모집단의 모수를 추정해야 하는 상황을 생각해봅시다.

```
In [23]: from scipy.stats import norm

np.random.seed(123)
plt.figure(figsize=(5, 3))
x1 = norm.rvs(loc = -10, scale=3, size=300)
x2 = norm.rvs(loc = 10, scale=2, size=550)
X = np.hstack([x1, x2])
```

```
sns.histplot(X, bins=30)
plt.show()
```



x는 두 개의 정규 분포가 섞여 있는 분포 입니다.

두 개의 정규 분포의 모수인 평균과 분산을 추정하기 위해서는

각 표본들이 어떤 분포에 속한지 알아야 합니다.

하지만 모수를 알 지 못한다면,

표본이 어떤 분포에 속하는지를 알 수 없는 상황이 됩니다.

그렇기 때문에 평균과 분산을 추정하기도 어려워집니다.

```
In [30]: Image('138.png')
```

Out[30]: **Expectation Maximization(EM) 알고리즘**

### 아이디어

일단,  $\theta$  임의의 수로 정의한후  $Z$ (미지의 변수, latent variables)를 구합니다.

$Z$ 와  $X$ 를 가장 잘 설명하는(최대 우도를 갖는)  $\theta$ 를 구하고, 이  $\theta$ 로 다시  $Z$ 를 구하는 일련의 과정을 반복합니다.

### 알고리즘

1.  $\theta$ 를 초기화 합니다.
2. 기대값 단계(Expectation, E step)

$Q(\theta'|\theta)$ : 현재 단계 모수  $\theta$ 에서 다음 단계 모수  $\theta'$ 에 대한 로그 우도  $\log(p(X, Z|\theta'))$ 의 기대값입니다.

$$Q(\theta'|\theta) = E_{Z \sim p(\cdot|X, \theta)} [\log P(X, Z|\theta')]$$

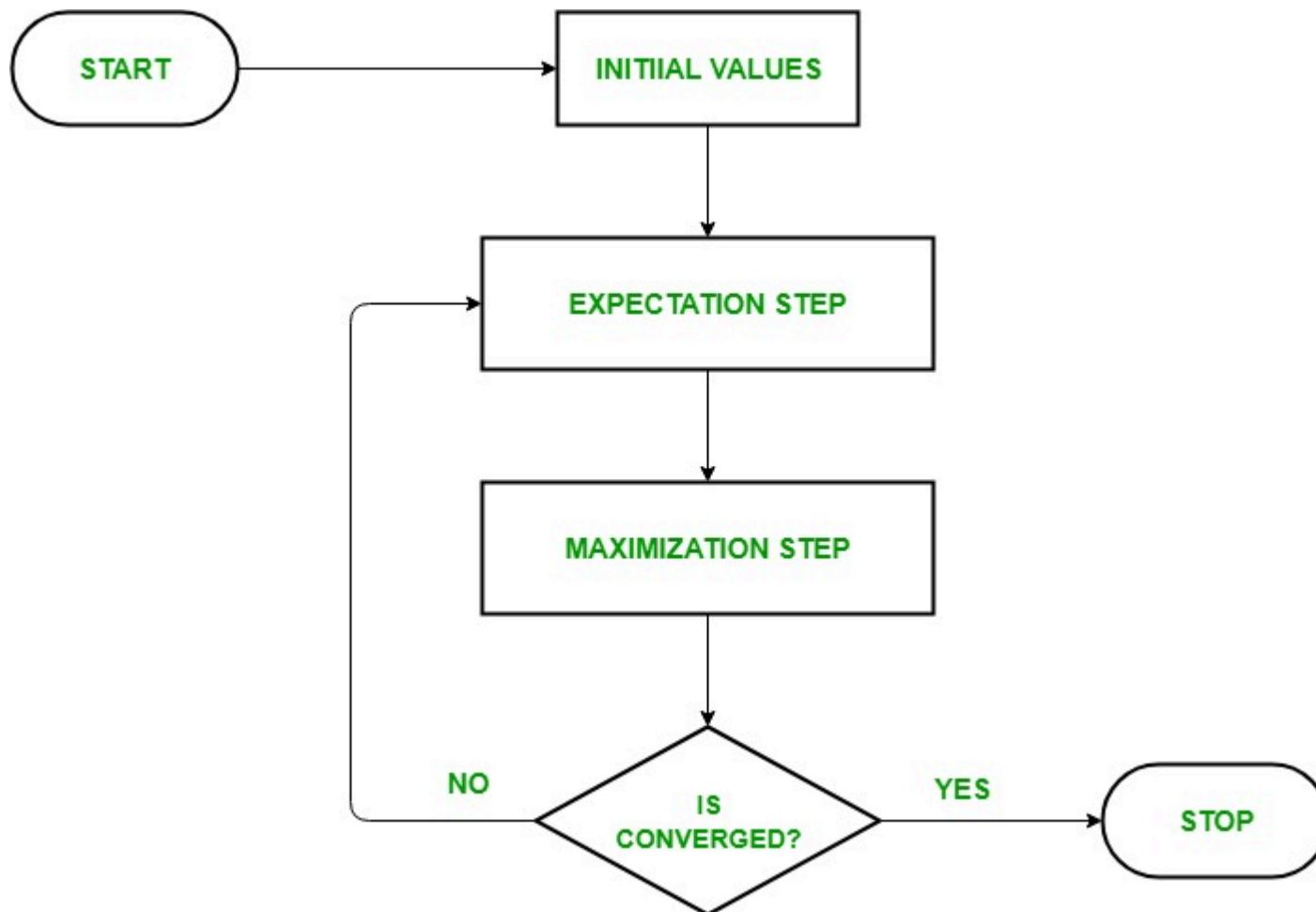
3. 최대화 단계(Maximization, M step)

$$\theta' = \operatorname{argmax}_{\theta'} Q(\theta'|\theta)$$

4.  $\theta = \theta'$  로 업데이트하고 수렴할 때까지 2부터 반복합니다.

In [34]: `Image('EM.jpg')`

Out[34]:



In [35]: Image('139.png')

Out[35]: **[Ex.7]**

위에서 생성한 표본을 생성한 분포의 모수를 EM 알고리즘으로 구하기 위해  $Q(\theta'|\theta)$ 를 최대화하는  $\theta'$ 을 구해봅니다.

In [32]: Image('140.png')

Out[32]:  $Q(\theta'|\theta)$ 을 구하기 위해 먼저  $Z \sim p(\cdot|X, \theta)$  를 구해 봅니다.

### 기대값(Expectation) 단계

$$T_{j,i} = P(Z_i = j|X_i = x_i; \theta)$$

$$T_{j,i} = \frac{P(X_i=x_i|Z_i=j;\theta)P(Z=j)}{P(X_i=x_i;\theta)}$$

$$T_{j,i} = \frac{P(X_i=x_i|Z_i=j;\theta)P(Z=j)}{\sum_{k=1}^C P(X_i=x_i|Z_i=k;\theta)P(Z=k)}$$

$$X_i|Z_i = 1 \sim N(\mu_1, \sigma_1^2), \dots, X_i|Z_i = C \sim N(\mu_c, \sigma_c^2)$$

$$P(Z = 1) = \tau_1, \dots, P(Z = C) = \tau_c$$

$X$ 의 분포와  $Z$ 의 분포를 정의하기 위해 필요한 모수는  $\theta = (\mu, \sigma, \tau)$  입니다.

$$Q(\theta'|\theta) = \sum_{i=1}^n \sum_{j=1}^C T_{ji} (\log P(X_i = x_i, Z_i = j|\theta'))$$

$$\log P(X_i = x_i, Z_i = j|\theta') = \log(P(X_i = x_i|Z_i = j; \theta) + \log P(Z = j; \theta'))$$

$$= \log\left(\frac{1}{\sqrt{2\pi}\sigma_j}\right) - \frac{1}{2}\left(\frac{x_i - \mu_j}{\sigma_j}\right)^2 + \log \tau_j$$

### 최대화(Maximization) 단계

$$Q(\theta'|\theta) = \sum_{i=1}^n \sum_{j=1}^C T_{j,i} \left( \log\left(\frac{1}{\sqrt{2\pi}\sigma_j}\right) - \frac{1}{2}\left(\frac{x_i - \mu_j}{\sigma_j}\right)^2 + \log \tau_j \right)$$

$$\tau' = \operatorname{argmax}_{\tau} \sum_{i=1}^n \sum_{j=1}^C T_{j,i} \log \tau_j$$

$$\tau'_j = \frac{1}{n} \sum_{i=1}^n T_{j,i}$$

$$(\mu', \sigma') = \operatorname{argmax}_{\mu, \sigma} \sum_{i=1}^n \sum_{j=1}^C T_{j,i} \left( -\log(\sigma_j) - \frac{1}{2} \left( \frac{x_i - \mu_j}{\sigma_j} \right)^2 \right)$$

$$\mu'_j = \frac{\sum_{i=1}^n T_{j,i} x_i}{\sum_{i=1}^n T_{j,i}},$$

$$\sigma_j'^2 = \frac{\sum_{i=1}^n T_{j,i} (x_i - \mu_j)^2}{\sum_{i=1}^n T_{j,i}}$$

**[Ex.8]**

[Ex.7]에서 도출한 내용들로 두 개의 정규 분포의 모수를

EM 알고리즘을 통해 점추정해봅시다.

In [24]:

```
'''
가우시안 혼합 모델(Gaussian Mixture Model, GMM)은
확률적 생성 모델 중 하나로, 여러 개의 가우시안 분포의
선형 조합으로 데이터를 모델링하는 확률 모델입니다.
GMM은 비지도 학습 방법으로,
데이터의 군집화(clustering)나 데이터 생성에 주로 사용됩니다.

GMM은 데이터가 여러 개의 가우시안 분포에서 생성된다고 가정합니다.
각 가우시안 분포는 데이터의 특정 하위 그룹을 나타내며,
이러한 가우시안 분포들은 서로 다른 평균과 분산을 가집니다.
따라서 GMM은 데이터를 여러 개의 서로 다른 클러스터로
나누는 군집화(clustering) 알고리즘으로 사용될 수 있습니다.

GMM은 EM(Expectation-Maximization) 알고리즘을 사용하여 학습됩니다.
EM 알고리즘은 각 데이터 포인트가 주어졌을 때
가우시안 분포의 파라미터를 업데이트하는 과정을 반복하여
최적의 파라미터를 찾아냅니다.
EM 알고리즘은 "Expectation(기댓값)" 단계에서
각 데이터 포인트가 각 클러스터에 속할 확률을 추정하고,
```

"Maximization(최대화)" 단계에서 이 확률을 기반으로 가우시안 분포의 파라미터를 업데이트합니다.

GMM은 다음과 같은 주요 특징을 가지고 있습니다:

1. 유연성: 다양한 데이터 분포를 모델링할 수 있습니다.
2. 클러스터링: 데이터를 여러 개의 클러스터로 군집화할 수 있습니다.
3. 확률적 모델링: 각 데이터 포인트가 각 클러스터에 속할 확률을 제공합니다.
4. 모델 복잡도 조절: 가우시안 분포의 수를 조절하여 모델의 복잡도를 조절할 수 있습니다.

GMM은 데이터의 분포가 가우시안 분포를 따른다는 가정하에 동작하기 때문에, 데이터가 실제로 가우시안 분포를 따를 때 가장 효과적으로 동작합니다.

...

코드는 가우시안 혼합 모델(Gaussian Mixture Model, GMM)의 모수를 초기화하는 과정을 나타냅니다.

이 과정은 EM(Expectation-Maximization) 알고리즘을 사용하여 GMM을 학습하기 전에 필요합니다.

각 변수는 다음과 같은 역할을 합니다:

1. ``n_mixture = 2``: 혼합 모델에서 사용할 가우시안 분포의 수를 결정합니다. 이 경우 2개의 가우시안 분포를 사용합니다.
2. ``n = X.shape[0]``: 데이터 포인트의 개수를 나타냅니다.
3. ``mu = np.random.uniform(-3, 3, size=n_mixture)``: 각 가우시안 분포의 평균(`mu`)을 초기화합니다. ``np.random.uniform(-3, 3, size=n_mixture)``는 균일 분포에서 -3부터 3까지의 무작위 값 생성합니다. 이 값을 가우시안 분포의 평균에 더하여 초기화합니다.
4. ``mu = mu + X.mean()``: 데이터의 평균을 추가하여 초기화된 `mu` 값을 조정합니다. 이렇게 함으로써 초기 평균이 데이터의 분포에 가깝도록 합니다.
5. ``sigma = np.ones(n_mixture)``: 각 가우시안 분포의 표준편차(`sigma`)를 초기화합니다. 모든 가우시안 분포의 표준편차를 1로 설정합니다.
6. ``tau = np.ones(n_mixture) / n_mixture``:

각 가우시안 분포의 혼합 계수( $\tau$ )를 초기화합니다.  
초기에는 각 가우시안 분포가 모든 데이터 포인트에 대해 동일한 가중치를 가지도록 합니다.  
따라서 혼합 계수를 모두 동일한 값으로 설정하고,  
전체 합이 1이 되도록 정규화합니다.

이러한 초기화는 EM 알고리즘이 시작될 때  
모델을 초기화하기 위해 필요합니다.  
EM 알고리즘은 데이터에 적합한 가우시안 분포들을  
찾아내는 과정으로, 초기화된 모수를 사용하여 시작됩니다.  
'''

# 모수를 초기화 합니다.

```
n_mixture = 2
n = X.shape[0]

mu = np.random.uniform(-3, 3, size=n_mixture)
mu = mu + X.mean()

sigma = np.ones(n_mixture)
tau = np.ones(n_mixture) / n_mixture
```

In [25]:

```
'''
코드는 EM 알고리즘의 "Expectation(기댓값)" 단계에서
각 데이터 포인트가 속하는 각 분포의 확률을
계산하는 과정을 나타냅니다.

1. `norm.pdf(X, loc=mu[i], scale=sigma[i])`:
 각 데이터 포인트 `X`가 각 분포에서 나타날 확률을 계산합니다.
 이때 `norm.pdf()` 함수는 정규 분포의 확률 밀도 함수(PDF)를
 계산하는 함수입니다. `loc`은 정규 분포의 평균을 나타내며,
 `scale`은 표준편차를 나타냅니다.
 각각의 데이터 포인트 `X`에 대해
 `n_mixture`개의 정규 분포에서의 확률을 계산합니다.

2. `T = np.vstack(...)` :
 계산된 각 데이터 포인트의 확률을 수직 방향으로 쌓아서
 행렬 `T`를 생성합니다. 이 행렬의 각 행은 한 데이터 포인트에
 대해 각 분포에서의 확률을 나타냅니다.

3. `T = T / np.sum(T, axis=1, keepdims=True)` :
```

```

 각 데이터 포인트가 속하는 각 분포의 확률을
 전체 확률의 합으로 나누어 정규화(normalization)합니다.
 이를 통해 각 데이터 포인트가 속하는 각 분포에 대한
 조건부 확률을 얻을 수 있습니다.
 따라서 `T_ij`는 데이터 포인트 `i`가 분포 `j`에 속할
 조건부 확률을 나타냅니다.
 ...

```

```

from scipy.stats import norm

```

```

Expectation 관계
T_ij: 각 데이터 포인트가 속하는 분포의 확률을 구합니다.
T = np.vstack([norm.pdf(X, loc = mu[i], \
 scale = sigma[i]) \
 for i in range(n_mixture)]).T
T = T / np.sum(T, axis=1, keepdims=True)

```

In [26]:

```

'''
코드는 EM 알고리즘의 "Maximization(최대화)" 단계 중
첫 번째 단계에서 각 혼합 요소의 가중치인
tau를 업데이트하는 과정을 나타냅니다.

1. `T_sum = T.sum(axis=0)` :
 행렬 `T`의 각 열을 합하여 각 분포에 속하는 데이터 포인트의
 합을 구합니다. 이렇게 하면 각 분포에 속하는
 데이터 포인트의 총 수를 나타내는 배열 `T_sum`이 생성됩니다.
2. `tau_prime = T_sum / n` :
 `T_sum`을 데이터 포인트의 총 수 `n`으로 나누어 각 분포에
 속하는 데이터의 비율을 계산합니다.
 이렇게 하면 각 분포의 가중치를 나타내는 새로운 tau 값인
 `tau_prime`이 생성됩니다. 각 tau_prime 값은 해당 분포에
 속하는 데이터의 비율을 나타냅니다.
 ...

```

```

Maximization 단계 1: tau
tau_prime을 구합니다.
T_sum = T.sum(axis=0)
tau_prime = T_sum / n
tau_prime

```

Out[26]: array([0.35297322, 0.64702678])

In [27]:

```
'''
코드는 EM 알고리즘의 "Maximization(최대화)" 단계 중
두 번째 단계에서 각 혼합 요소의 평균(mu)과
표준 편차(sigma)를 업데이트하는 과정을 나타냅니다.

1. `T = T / T_sum`:
 앞서 계산한 `T` 행렬에서 각 열의 값을 해당 열의 총 합
 `T_sum`으로 나누어서 각 데이터 포인트가 속하는
 혼합 요소에 대한 확률을 계산합니다.
 이렇게 하면 각 데이터 포인트의 가중치가 적용된
 새로운 확률 행렬 `T`가 생성됩니다.

2. `mu_prime
 = np.array([np.dot(T[:, i], X)
 for i in range(n_mixture)])`:
 가중 평균을 사용하여 각 혼합 요소의 평균(mu)을
 업데이트합니다. `T` 행렬의 각 열은 각 데이터 포인트가
 해당 혼합 요소에 속할 확률을 나타냅니다.
 이를 사용하여 가중 평균을 계산하고,
 각 혼합 요소에 대한 새로운 평균 값
 `mu_prime`을 계산합니다.

3. `sigma_prime
 = np.array([np.dot(T[:, i],
 np.square(X - mu_prime[i])) ** 0.5
 for i in range(n_mixture)])`:
 업데이트된 `T` 행렬을 사용하여 각 혼합 요소의
 표준 편차(sigma)를 업데이트합니다.
 각 데이터 포인트가 해당 혼합 요소에 속할 확률을
 사용하여 가중 표준 편차를 계산하고,
 이를 각 혼합 요소에 대한 새로운 표준 편차 값
 `sigma_prime`으로 계산합니다.
'''

Maxmization 단계2: mu, sigma
T = T / T_sum
mu_prime = np.array([np.dot(T[:, i], X) \
 for i in range(n_mixture)])
sigma_prime = np.array([np.dot(T[:, i],
 np.square(X - mu_prime[i])) ** 0.5
```

```

 for i in range(n_mixture)])

mu_prime, sigma_prime

```

Out[27]: (array([-10.08623236, 9.95619206]), array([3.08263718, 1.98215196]))

### [Ex.9]

[Ex.8]를 기반으로 EM 알고리즘의 업데이트 과정을

initial, 3, 6, 9 번 했을 때의 학습 진행 과정을 시각화해봅니다.

```

In [28]: '''
코드는 EM(Expectation-Maximization) 알고리즘을 사용하여
Gaussian Mixture Model(GMM)을 추정하고
그 과정을 시각화하는 과정을 나타냅니다.

1. `plot_dist` 함수:
 EM 알고리즘의 진행 과정을 시각화하는 함수입니다.
 입력으로는 matplotlib의 서브플롯(ax), 변수(X),
 평균(mu), 표준 편차(sigma), 그리고 각 데이터 포인트가
 각 혼합 요소에 속할 확률(T)을 받습니다.
 이 함수는 주어진 데이터 X의 히스토그램과 함께
 각 혼합 요소의 정규 분포를 플로팅합니다.

2. 모수 초기화:
 초기에는 두 개의 혼합 요소를 가진
 Gaussian Mixture Model을 설정하고,
 평균(mu)은 임의로 선택되며 평균 값의 평균으로
 초기화되고, 표준 편차(sigma)는 모든 혼합 요소에
 대해 1로 초기화됩니다.

3. EM 알고리즘 반복:
 EM 알고리즘의 "Expectation"과 "Maximization" 단계를
 반복적으로 수행합니다.
 "Expectation" 단계에서는 각 데이터 포인트가 각 혼합 요소에
 속할 확률을 계산하고, "Maximization" 단계에서는
 이를 사용하여 혼합 요소의 파라미터를 업데이트합니다.
 이를 반복하면서 혼합 요소의 수렴된 파라미터를 얻게 됩니다.
'''

```

## 4. 시각화:

EM 알고리즘의 반복을 통해 각 단계에서의 혼합 모델을 시각화합니다. 초기화된 혼합 모델과 몇 단계의 업데이트된 혼합 모델을 함께 플로팅하여 각 단계에서 모델이 어떻게 변하는지 관찰할 수 있습니다.

...

...

코드를 더 자세히 설명해 보겠습니다.

1. `plot_dist` 함수 정의:

- `plot_dist` 함수는 EM(Expectation-Maximization) 알고리즘의 진행 단계를 시각화하기 위한 함수입니다.
- 주어진 데이터 `X`의 히스토그램을 그리고, 각 혼합 요소의 정규 분포 곡선을 함께 표시합니다.
- 서브플롯 `ax`, 변수 `X`, 평균 `mu`, 표준 편차 `sigma`, 각 데이터 포인트가 각 혼합 요소에 속할 확률 `T`을 입력으로 받습니다.

## 2. 모수 초기화:

- `n_mixture` 개수의 혼합 요소를 가진 Gaussian Mixture Model을 설정합니다.
- `mu`는 각 혼합 요소의 평균을 나타내는 배열로, 초기 값은 `-3`에서 `3` 사이의 균등 분포에서 무작위로 선택된 값에 데이터의 평균을 더하여 초기화됩니다. 이렇게 함으로써 초기 평균은 데이터의 분포에 가깝게 설정됩니다.
- `sigma`는 각 혼합 요소의 표준 편차를 나타내는 배열로, 모든 요소의 표준 편차를 1로 초기화합니다.

## 3. EM 알고리즘 반복:

- EM 알고리즘의 "Expectation"과 "Maximization" 단계를 반복하여 모델을 학습합니다.
- "Expectation" 단계에서는 주어진 데이터 포인트가 각 혼합 요소에 속할 확률을 계산합니다. 이는 데이터 포인트가 각 혼합 요소의 확률 밀도 함수에서 발생할 확률로 계산됩니다.
- "Maximization" 단계에서는 "Expectation" 단계에서 계산된 데이터 포인트의 소속 확률을 사용하여 각 혼합 요소의 파라미터를 업데이트합니다. 이 단계에서는 각 혼합 요소의 평균 `mu`와 표준 편차 `sigma`를 업데이트합니다.

## 4. 시각화:

- EM 알고리즘의 반복을 통해 각 단계에서의 혼합 모델을 시각화합니다.
- 초기화된 혼합 모델과 몇 단계의 업데이트된 혼합 모델을 함께 플로팅하여 모델의 변화를 관찰할 수 있습니다. 이를 통해 모델이 데이터에 잘 적합되는 방향으로 업데이트되는 과정을 시각적으로 이해할 수 있습니다.

...

...

```
sigma_prime \
= np.array([np.dot(T[:, i], \
 np.square(X- mu_prime[i])) ** 0.5 \
 for i in range(n_mixture)])
```

위 코드는 EM 알고리즘의 Maximization 단계에서 각 혼합 요소의 표준 편차(`sigma`)를 업데이트하는 부분입니다.

#### 1. for 루프:

- `for i in range(n\_mixture)`:  
주어진 데이터에 대해 각 혼합 요소(`i`)에 대해 반복합니다.

#### 2. 표준 편차 계산:

- `np.square(X - mu\_prime[i])`:  
주어진 데이터와 해당 혼합 요소의 평균 간의 차이를 제공합니다. 이렇게 함으로써 각 데이터 포인트와 해당 혼합 요소 간의 거리의 제곱을 계산합니다.
- `np.dot(T[:, i], ... )`:  
각 데이터 포인트가 혼합 요소 `i`에 속할 확률 `T[:, i]`과 앞서 계산한 거리의 제곱을 내적합니다.  
이는 각 데이터 포인트의 거리를 확률 가중치로 고려하여 평균 거리를 계산하는 것과 같습니다.
- `0.5`:  
이를 통해 각 혼합 요소에 대한 표준 편차를 구합니다. 거리의 제곱을 계산했기 때문에 제곱근을 취하여 표준 편차를 얻습니다.
- `np.array([...])`:  
각 혼합 요소에 대한 표준 편차를 배열로 저장합니다.

#### 3. 결과:

- 최종적으로 `sigma\_prime`에는 각 혼합 요소에 대한 새로운 표준 편차가 저장됩니다.
- 이렇게 함으로써 각 혼합 요소의 표준 편차가 업데이트되며, EM 알고리즘의 Maximization 단계가 완료됩니다.

```

'''
from scipy.stats import norm

def plot_dist(ax, X, mu, sigma, T):
 """
 EM 알고리즘의 진행단계를 시각화합니다.
 Parameters
 ax: ax
 출력한 ax
 X: np.ndarray
 변수
 mu: np.ndarray
 평균
 T: np.array
 표본별 확률 분포 별 속할 확률
 """
 x_range = np.linspace(-15, 15, 100)
 sns.histplot(x=X, hue=np.argmax(T, axis=1), bins=30, ax=ax)
 ax.set_title('μ: {}, σ: {}'.format(np.round(mu, 2), np.round(sigma, 2)))
 tax = ax.twinx()
 tax.set_ylim(0, 0.2)
 for i in range(mu.shape[0]):
 tax.plot(x_range, norm.pdf(x_range,
 loc=mu[i],
 scale=sigma[i]))

모수를 초기화 합니다.
n_mixture = 2
n = X.shape[0]

mu = np.random.uniform(-3, 3, size=n_mixture)
mu = mu + X.mean()

sigma = np.ones(n_mixture)
tau = np.ones(n_mixture) / n_mixture

fig, axes = plt.subplots(1, 4, figsize=(16, 4))

plot_dist(axes[0], X, mu, sigma, T)

```

```

cnt = 1
for it in range(10):
 T = np.vstack([norm.pdf(X, loc = mu[i], \
 scale = sigma[i]) \
 for i in range(n_mixture)]).T
 T = T / np.sum(T, axis=1, keepdims=True)
 T_sum = T.sum(axis=0)
 tau_prime = T_sum / n
 T = T / T_sum
 mu_prime = np.array([np.dot(T[:, i], X) \
 for i in range(n_mixture)])

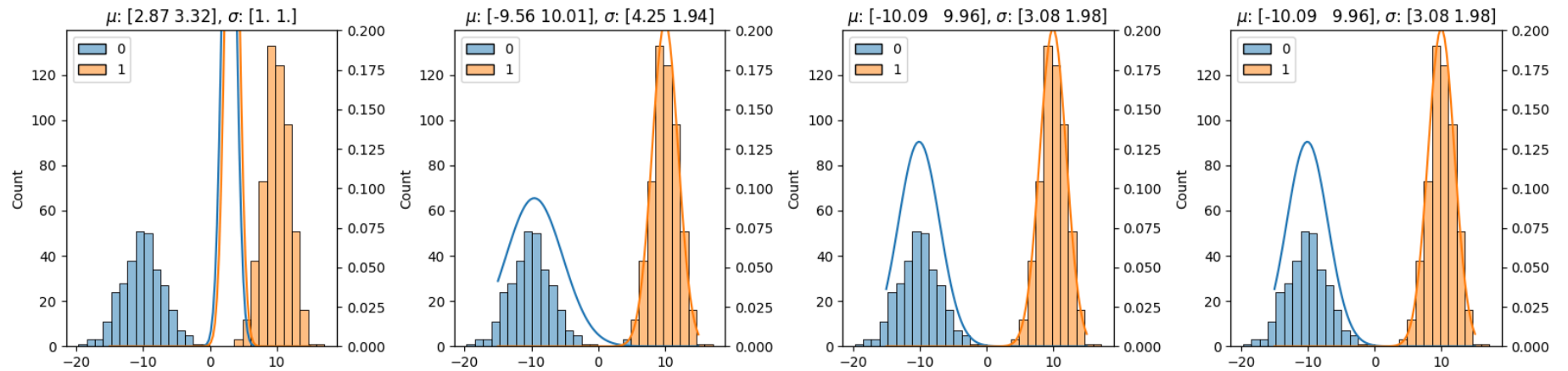
 sigma_prime \
 = np.array([np.dot(T[:, i], \
 np.square(X - mu_prime[i])) * 0.5 \
 for i in range(n_mixture)])

 tau, mu, sigma = tau_prime, mu_prime, sigma_prime

 if it in [1, 3, 9]:
 plot_dist(axes[cnt], X, mu, sigma, T)
 cnt += 1

plt.tight_layout()
plt.show()

```



### 3. MCMC(Markov Chain Monte Carlo)

MCMC는 확률적 표본추출 기법 중의 하나로, 목표 분포에서 표본을 추출합니다.

마르코프 체인에서의 이전 상태와의 독립성과 Monte Carlo의 무작위 시도를 통한

산출 방법을 응용하여 복잡한 확률 분포의 표본을 추출하는 방법들을 포괄적으로 일컫습니다.

통계와 머신러닝에서는 모수를 샘플링하여 모수를 추정에 사용합니다.

MCMC의 알고리즘의 한 예로 메트로폴리스-헤이스팅스

알고리즘(Metropolis-Hastings) 이 있습니다.

메트로폴리스-헤이스팅스 알고리즘은 주어진 모수의 사전(Prior) 분포에서

관측 데이터에 대한 모수의 사후(Posterior)의 분포에서 모수를 추출하여,

사후 분포 상의 모수를 추정하는 것에 사용합니다.

## 몬테 카를로 추정(Monte Carlo Estimation)

- 무작위 시도의 반복을 통하여 대상값을 도출합니다.
- 특히 해석적으로 풀이가 거의 불가능한 문제의 값을 구합니다.

```
In [36]: Image('141.png')
```

Out[36]: **[Ex.10]**

반지름 1인 원의 넓이를 Monte Carlo 방법을 통해 구해봅니다.

반지름이 1인 원의 넓이:  $\pi = 3.141592653589793...$

2×2 정사각형에 임의의 점을 찍을 때 원 안에 위치할 확률을 p라고 하면

원의 넓이  $\approx p \cdot 4$ (2×2 정사각형 넓이)

In [37]: `Image('142.png')`

Out[37]: 이 코드는 몬테 카를로 시뮬레이션(Monte Carlo simulation)을 사용하여 원의 넓이를 추정하는 방법을 구현합니다.

### 1. 원 안에 위치할 확률 계산:

- `'x'`와 `'y'`는 -1에서 1 사이의 균일한 분포에서 무작위로 선택된 점들을 나타냅니다.
- 이 점들의 원 안에 위치할 확률을 계산하기 위해, 원의 방정식을 사용합니다. 원의 방정식은  $x^2 + y^2 = r^2$ 입니다. 여기서 원의 반지름  $r = 1$ 입니다.
- 따라서  $x^2 + y^2$ 가 1보다 작거나 같으면 해당 점은 원 안에 위치합니다. 이를 나타내기 위해 `'is_in_circle'` 배열을 만듭니다.

### 2. 원의 넓이 추정:

- `'is_in_circle'` 배열에서 `'True'`인 값의 개수를 세고, 전체 시도 횟수인 `'n_trial'`로 나누어 원 안에 위치할 확률  $p$ 를 계산합니다.
- 원의 넓이는 반지름이 1인 원이므로, 원의 넓이를  $4 \times p$ 로 추정할 수 있습니다. 이는 전체 정사각형 영역(1x1) 내에서 원 안에 위치할 확률에 4를 곱한 값과 같습니다.

따라서 코드의 마지막 줄은 추정된 원의 넓이를 출력합니다.

```
In [29]: cnt = 0
n_trial = 10000

x = np.random.uniform(-1, 1, size=n_trial)
y = np.random.uniform(-1, 1, size=n_trial)
원 안에 위치할 확률
```

```
is_in_circle = np.sqrt(x ** 2 + y ** 2) <= 1

p = is_in_circle.sum() / n_trial
p * 4
```

Out[29]: 3.1508

In [38]: Image('143.png')

Out[38]: **마르코프 체인(Markov Chain)**

시간이나 공간의 흐름에 따라 특정 상태(state)로의 전이를 모델링하는 수학적 도구입니다.

이는 확률적인 시스템의 행동을 분석하고 예측하는 곳에 사용됩니다.

시점(또는 지점)마다 상태를 가지고 있고, 각 상태는 일정한 확률로 전이될 수 있습니다.

- 상태 공간(State Space): 시스템이 취할 수 있는 가능한 모든 상태들을 나타냅니다.
- 전이 확률(Transition Probability): 한 상태에서 다른 상태로 전이될 확률을 나타냅니다.

전이 확률 행렬: 전이 확률을 요소로 담고 있는 행렬입니다.

$$P = \begin{bmatrix} P_{11} & P_{12} & \cdots & P_{1n} \\ P_{21} & P_{22} & \cdots & P_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ P_{n1} & P_{n2} & \cdots & P_{nn} \end{bmatrix}, P_{ij}: i \text{ 상태에서 } j \text{ 상태로 전이될 확률입니다.}$$

임의의 j에 대해서  $\sum_{i=1}^n P_{ji} = 1$  입니다.

상태 확률을 행벡터  $v$ 라고 하면, 다음 상태 확률을  $v'$ 라고 하면  $v' = vP$

- 마르코프 속성(Markov Property): 다음 상태로 전이는 오직 현재 상태하고만 연관이 있습니다. 메모리리스(memoryless)

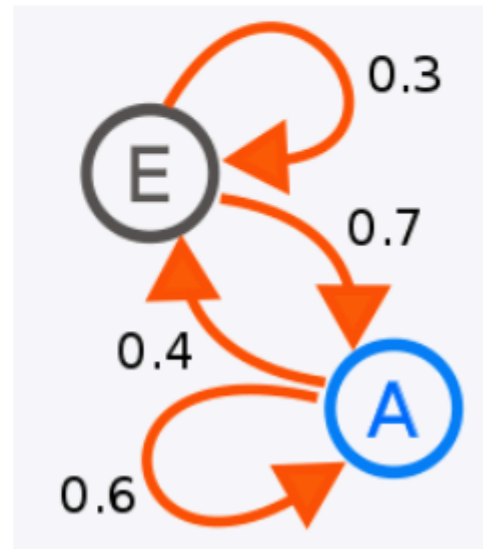
In [39]: `Image('144.png')`

Out[39]: **[Ex.11]**

상태 공간은 {E, A} 입니다. 아래 다이어그램은 E와 A사이의 전이 확률을 나타냅니다.

현재 E 상태일 확률은 0.7, A 상태일 확률이 0.3 일 때,

1에서 20단계가 지나후의 E 상태일 때와 A 상태일 확률을 구하여 그래프로 출력해봅니다.



※ 이미지 출처: [https://en.wikipedia.org/wiki/Markov\\_chain](https://en.wikipedia.org/wiki/Markov_chain)

In [39]: `'''`

이 코드는 마코프 체인(Markov chain)의 초기 상태와 전이 확률 행렬을 사용하여 시간이 지남에 따른 각 상태의 확률을 계산합니다.

1. 초기 상태 및 전이 확률 행렬 설정:

- ``v``는 초기 상태로서, 각 상태의 초기 확률을 나타내는 1차원 배열입니다.

예를 들어, 첫 번째 원소인 `v[0]`는 상태 E에 대한 초기 확률을, 두 번째 원소인 `v[1]`는 상태 A에 대한 초기 확률을 나타냅니다.  
 - `P`는 전이 확률 행렬로서, 각 행은 현재 상태에서 다음 상태로의 전이 확률을 나타냅니다.  
 여기서 `P[i, j]`는 상태 i에서 상태 j로의 전이 확률을 의미합니다.

### 2. 상태 변화 시뮬레이션:

- `v_list`는 각 단계에서의 상태 확률을 저장하는 리스트입니다. 초기 상태 `v`를 첫 번째 원소로 추가합니다.
- `v_list[-1]`에 현재 상태의 확률이 저장되어 있습니다. 이를 전이 확률 행렬 `P`와 행렬 곱셈하여 다음 단계에서의 상태 확률을 계산합니다.
- 계산된 다음 상태의 확률은 `v_list`에 추가됩니다.

### 3. 결과 시각화:

- `v_list`를 행렬로 변환하여 각 단계에서의 E와 A 상태의 확률을 저장합니다.
- `plt.plot()`을 사용하여 각 단계에서의 E와 A 상태의 확률을 시간에 따라 시각화합니다.

이렇게 함으로써 코드는 초기 상태와 전이 확률을 바탕으로 시간이 지남에 따른 각 상태의 변화를 추적하고 시각화합니다. 전이 확률이 정상 상태에 수렴할 때까지의 과정을 시뮬레이션한 것으로 보입니다.

처음에는 초기 확률 `v`에 따라 E와 A의 확률이 변화하지만, 시간이 흐름에 따라 전이 확률 행렬 `P`에 따라 두 상태 간의 변화가 일어납니다.

이런 변화가 반복되면서 상태 간의 확률이 수렴하게 되는데, 이것이 시뮬레이션 결과에서는 그래프가 어느 정도의 시간 이후에 일정한 값으로 수렴하는 것을 확인할 수 있습니다.

따라서 코드는 초기 상태와 전이 확률을 바탕으로 한 마코프 체인의 상태 변화를 표현하고 시간에 따른 상태 확률의 변화를 시각화한 것입니다.

...

# 초기 확률입니다.

```
v = np.array([0.7, 0.3])
```

# 전이 확률 행렬입니다.

```
P = np.array([[0.3, 0.7], [0.4, 0.6]])
```

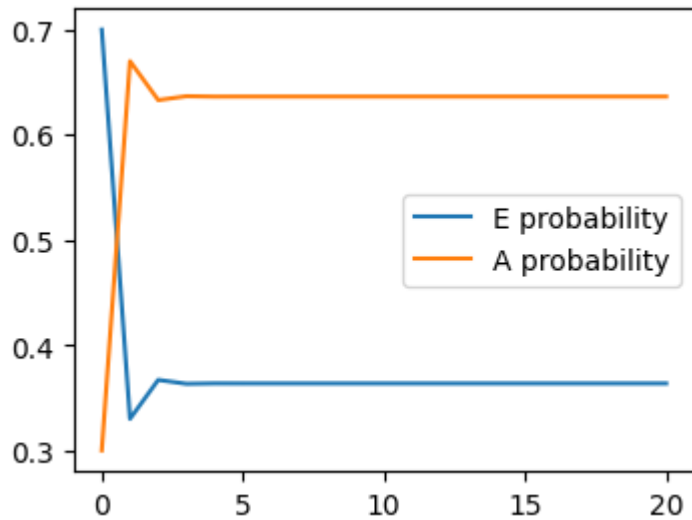
```
v_list = [v]
```

```

for _ in range(20):
 v_list.append(np.dot(v_list[-1], P))

v_list = np.vstack(v_list)
plt.figure(figsize=(4, 3))
plt.plot(v_list[:, 0], label='E probability')
plt.plot(v_list[:, 1], label='A probability')
plt.legend()
plt.show()

```



### [Ex. 12]

상태 공간은 {A, B} 입니다. A에서 B로 이동할 확률은 1.0이고,

B에서 A로 이동할 확률은 1.0입니다.

현재 A 상태일 확률은 0.7, B 상태일 확률이 0.3 일 때,

1에서 20단계가 지나후의 A 상태일 때와 B 상태일 확률을 구하여

그래프로 출력해봅니다.

In [31]:

```
'''
코드는 두 개의 상태(E와 A)가 있고, 전이 확률 행렬 `P`가
주어진 경우에 대한 마코프 체인의 초기 상태와
상태 변화를 시뮬레이션하는 것입니다.

1. 초기 상태 및 전이 확률 행렬 설정:
- `v`는 초기 상태로서, 각 상태의 초기 확률을 나타내는
 1차원 배열입니다. 예를 들어, 첫 번째 원소인 `v[0]`는
 상태 E에 대한 초기 확률을, 두 번째 원소인 `v[1]`는
 상태 A에 대한 초기 확률을 나타냅니다.
- `P`는 전이 확률 행렬로서, 각 행은 현재 상태에서
 다음 상태로의 전이 확률을 나타냅니다.
 여기서 `P[i, j]`는 상태 i에서 상태 j로의
 전이 확률을 의미합니다. 이 행렬은 현재의 상태와
 다음 상태 사이의 관계를 나타냅니다.

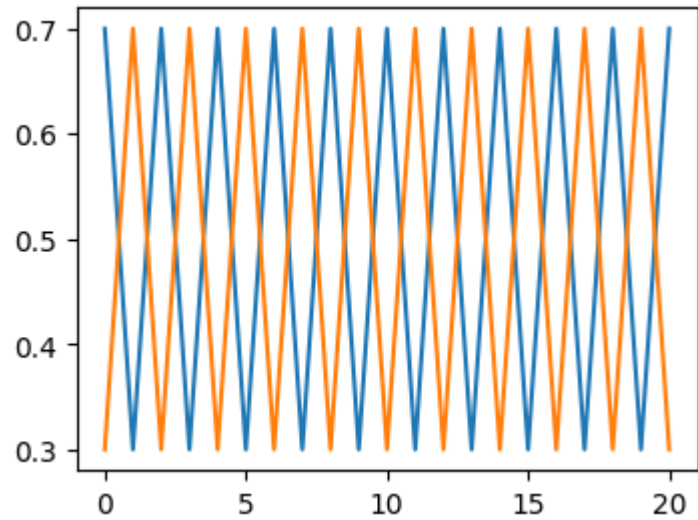
2. 상태 변화 시뮬레이션:
- `v_list`는 각 단계에서의 상태 확률을 저장하는
 리스트입니다. 초기 상태 `v`를 첫 번째 원소로 추가합니다.
- `v_list[-1]`에 현재 상태의 확률이 저장되어 있습니다.
 이를 전이 확률 행렬 `P`와 행렬 곱셈하여
 다음 단계에서의 상태 확률을 계산합니다.
- 계산된 다음 상태의 확률은 `v_list`에 추가됩니다.

3. 결과 시각화:
- `v_list`를 행렬로 변환하여 각 단계에서의
 E와 A 상태의 확률을 저장합니다.
- `plt.plot()`을 사용하여 각 단계에서의
 E와 A 상태의 확률을 시간에 따라 시각화합니다.

이렇게 함으로써 코드는 초기 상태와 전이 확률을 바탕으로 한
마코프 체인의 상태 변화를 표현하고
시간에 따른 상태 확률의 변화를 시각화합니다.
'''

초기 확률입니다.
v = np.array([0.7, 0.3])
P = np.array([[0.0, 1.0], [1.0, 0]])
v_list = [v]
for _ in range(20):
 v_list.append(np.dot(v_list[-1], P))
```

```
v_list = np.vstack(v_list)
plt.figure(figsize=(4, 3))
plt.plot(v_list[:, 0])
plt.plot(v_list[:, 1])
plt.show()
```



In [40]: `Image('145.png')`

Out[40]: **메트로폴리스-헤이스팅스(Metropolis-Hastings) 알고리즘**

MCMC 알고리즘에서 가장 기본적이면서 널리 사용되는 알고리즘 중 하나입니다.

목표 분포  $f(\theta)$ 로 부터 표본을 추출합니다.

### 알고리즘

$i$ 번째의 표본을  $\theta_i$ 라 정의합니다.

1. 시작 상태 설정: 상태 공간에 임의의 상태로 시작 합니다. ( $\theta_0$  추출)
2. 표본 제안(Sample Proposal): 현재 상태에서 새로운 상태를 제안합니다  $\theta_{i+1}$ 를 제안 받습니다.
3. 수용 확률(Acceptance Probability): 제안한 상태( $\theta_{i+1}$ )와 현재 상태( $\theta_i$ )의 목표 분포를 구합니다.

다음 표본이 수용될 확률을 현재 상태만을 고려하여 구한다는 점이 Markov Chain을 이용한 점입니다.

$$\alpha = \frac{f(\theta_{i+1})}{f(\theta_i)}$$

$\alpha \geq 1$  이면 무조건 수용

$\alpha < 1$  이면 몬테 카를로 기법을 응용하여  $[0, 1]$  사이의 난수를 생성하여 이 숫자가  $\alpha$  보다 작으면 수용

4. 현재 수용된 표본을 취하고, 목표한 표본을 얻을 때까지 2를 반복합니다.

In [41]: `Image('146.png')`

Out[41]: **모수의 사후 분포에서 모수 추출**

모수의 사전(Prior) 분포와 관측 데이터에서 모수의 사후(Posterior) 분포에서 모수를 추출합니다.

즉, 사전에 알고 있는 모수에 대한 지식을 바탕으로 관측 데이터를 통해 얻어지는 모수의 분포를 추정해봅니다.

베이즈 규칙을 이용하여 사전 분포와 사후 분포의 관계를 식으로 나타내면 아래와 같습니다.

$$p(\theta' | X, \theta) = \frac{P(X=x_1, \dots, X=x_n | \theta) P(\theta)}{P(X=x_1, \dots, X=x_n)}$$

$P(X = x_1, \dots, X = x_n)$ 를 구하는 것은 굉장히 어려운 일이기 때문에, 모수의 사후 확률 분포를 직접 계산하는 것 또한 어렵습니다.

메트로폴리스-헤이스팅스를 적용하면, 모수의 사후 분포 모수를 추출할 수 있습니다.

아래 메트로폴리스-헤이스팅스의 수용확률에 사후 확률 분포를 적용을 하면,

$$\alpha = \frac{f(\theta_{t+1})}{f(\theta_t)}$$

이전 상태의 확률이나 이후 제안의 증거 확률은  $P(X = x_1, \dots, X = x_n)$ 으로 동일하기 때문에

$$\alpha = \frac{P(X=x_1, \dots, X=x_n | \theta') P(\theta')}{P(X=x_1, \dots, X=x_n | \theta) P(\theta)}$$

로 나타내어 증거확률을 계산할 필요가 없게 됩니다.

이러한 원리로 사후 확률 계산에서의 난제인 증거 확률 계산 문제를 해소할 수 있습니다.

In [42]: `Image('147.png')`

Out[42]: **[Ex.13]**

아래 표본들은 정규분포에서 추출된 표본입니다. 이 표본들을  $X$ 라고 합니다. 각 표본들은 독립입니다

$X = [2.83, 6.99, 5.57, 1.99, 3.84, 8.3, 0.15, 4.14, 7.53, 3.27,$   
 $3.64, 4.81, 7.98, 3.72, 4.11, 4.13, 9.41, 9.37, 7.01, 5.77]$

이 정규분포의 평균 ( $\mu$ )의 사전(Prior) 분포는  $N(3, 4)$ 이고 분산( $\sigma^2$ )의 사전 분포는  $N(5, 2)$ 입니다.

목표 분포 함수는 아래와 같이 유도됩니다.

$$p(\mu', \sigma' | X, \mu, \sigma) \propto P(X = x_1, \dots, X = x_n | \mu, \sigma) P(\mu, \sigma) = \prod_{i=1}^n P(X = x_i | \mu, \sigma) P(\mu, \sigma)$$

사후 확률 분포의 모수  $\mu', \sigma'$ 를 추정하기 위해,

$\mu', \sigma'$ 을 메트로폴리스-헤이스팅스 알고리즘을 사용하여 5000개 추출해봅니다.

표본의 제안 방법은 아래와 같습니다.

$$\mu_{t+1} \sim N(\mu_t, 1)$$

$$\sigma_{t+1} \sim \text{Chi}^2(1) \times (\sigma_t)$$

수집한 10000개에서 표본 분포의 수렴점과 자기 상관성을 시각화를 통해 확인하여, 유효한 표본을 선택해봅니다.

In [32]:

```
'''
코드는 Metropolis-Hastings 알고리즘을 사용하여
주어진 목표 분포의 샘플을 생성하고 그 결과를 시각화합니다.

1. 목표 분포 설정:
- `target_log_distribution()` 함수는 목표 분포를 정의합니다.
이 함수는 입력으로 주어진 파라미터 `x`에 대해
로그 우도(log likelihood)와
사전 확률(prior)의 합을 반환합니다.
```

- 로그 우도는 각 데이터 포인트에 대한 정규 분포의 로그 확률을 계산합니다. 여기서 ``eps``는 로그에 0이 입력되는 것을 방지하기 위해 작은 수를 더해줍니다.
- 사전 확률은 각 파라미터가 사전에 설정된 평균과 분산을 따르는 정규 분포의 로그 확률을 계산합니다.

2. 제안 분포 설정:

- ``proposal_distribution()`` 함수는 현재 파라미터 ``x_current``를 기반으로 새로운 제안을 생성합니다. 여기서는 다변수 정규 분포를 사용합니다. 첫 번째 요소에 대해서는 정규 분포로, 두 번째 요소에 대해서는 카이 제곱 분포를 사용하여 생성합니다.

3. 수용 확률 계산:

- ``acceptance_probability()`` 함수는 새로운 제안 ``x_new``와 현재 상태 ``x_current`` 사이의 수용 확률을 계산합니다. 이는 Metropolis-Hastings 알고리즘의 핵심 부분입니다.
- 계산된 확률은 새로운 제안을 받아들일지 여부를 결정하는 데 사용됩니다.

4. Metropolis-Hastings 알고리즘 실행:

- ``metropolis_hastings()`` 함수는 주어진 반복 횟수에 따라 Metropolis-Hastings 알고리즘을 실행합니다. 각 반복에서 새로운 제안이 생성되고, 이를 현재 상태와 비교하여 수용 여부를 결정합니다. 결정된 상태는 샘플에 저장됩니다.

5. 결과 시각화:

- ``metropolis_hastings()`` 함수를 실행하여 얻은 샘플을 시각화합니다. 두 개의 그래프가 생성되며, 각각은 평균과 분산 파라미터에 대한 추이를 보여줍니다.

...

...

각 단계별 코드를 더 자세히 설명하겠습니다.

1. 목표 분포 설정:

- ``target_log_distribution()`` 함수는 주어진 파라미터 ``x``에 대해 로그 우도와 사전 확률의 합을 계산합니다.
- ``norm.pdf(X, loc=x[0], scale=x[1])``는 주어진 데이터 ``X``에 대해 정규 분포의 확률 밀도 함수 값을 계산합니다. 여기서 ``loc``은 평균이고 ``scale``은 표준편차입니다.
- ``np.log(norm.pdf(...))``는 각 데이터 포인트에 대한 로그 우도를 계산합니다. ``np.sum()``으로 모든 데이터 포인트의 로그 우도를 합산합니다.
- ``np.log(norm.pdf(x[0], loc=prior_mean_mu,`

```

scale=prior_mean_sigma 0.5))`는 첫 번째 파라미터에 대한
사전 확률을 계산합니다. 이는 정규 분포를 따르며,
`prior_mean_mu`는 평균이고, `prior_mean_sigma`는
표준 편차입니다.
- `np.log(norm.pdf(x[1], loc=prior_var_mu,
scale=prior_var_sigma 0.5))`는 두 번째 파라미터에 대한
사전 확률을 계산합니다. 이 역시 정규 분포를 따르며,
`prior_var_mu`는 평균이고, `prior_var_sigma`는
표준 편차입니다.
2. 제안 분포 설정:
- `proposal_distribution()` 함수는 현재 파라미터
`x_current`를 기반으로 새로운 제안을 생성합니다.
- 첫 번째 요소에 대해서는 정규 분포를 따르는 값을 생성하고,
두 번째 요소에 대해서는 카이 제곱 분포를 따르는
값을 생성합니다. 이때 `df=1`은 자유도를 1로 설정합니다.
3. 수용 확률 계산:
- `acceptance_probability()` 함수는 새로운 제안
`x_new`와 현재 상태 `x_current` 사이의
수용 확률을 계산합니다.
- `np.exp(target_log_distribution(x_new)
- target_log_distribution(x_current))`는
새로운 제안의 로그 우도와 현재 상태의 로그 우도 사이의
차이를 계산하고, 이를 확률로 변환합니다.

4. Metropolis-Hastings 알고리즘 실행:
- `metropolis_hastings()` 함수는 주어진 반복 횟수에 따라
Metropolis-Hastings 알고리즘을 실행합니다.
- 반복문에서는 각 반복에서 새로운 제안을 생성하고,
이를 현재 상태와 비교하여 수용 여부를 결정합니다.
결정된 상태는 샘플에 저장됩니다.

5. 결과 시각화:
- `metropolis_hastings()` 함수를 실행하여
얻은 샘플을 시각화합니다.
- 두 개의 그래프가 생성되며, 각각은 평균과 분산 파라미터에
대한 추이를 보여줍니다. 각 반복에서 파라미터 값이
어떻게 변하는지를 시각적으로 확인할 수 있습니다.
...

import numpy as np
import matplotlib.pyplot as plt

```

```

from scipy.stats import norm, chi2

X = [
 2.83, 6.99, 5.57, 1.99, 3.84, 8.3 , 0.15, 4.14, 7.53, 3.27,
 3.64, 4.81, 7.98, 3.72, 4.11, 4.13, 9.41, 9.37, 7.01, 5.77
]

prior_mean_mu, prior_mean_sigma = 3, 4
prior_var_mu, prior_var_sigma = 5, 2
log에 0이 입력되는 것을 차단하기 위해
임의의 작은 수를 정의합니다.
eps = 1e-7

목표 분포를 정의합니다.
계산 시 overflow를 예방하기 위해
log 확률을 사용하여 likelihood 계산합니다.
def target_log_distribution(x):
 # 로그 우도 계산합니다.
 log_likelihood = np.sum(np.log(norm.pdf(X,
 loc=x[0], scale=x[1]) + eps))
 # 사전 확률을 정규분포를 따르므로 이
 prior = np.log(norm.pdf(x[0], loc=prior_mean_mu,
 scale=prior_mean_sigma ** 0.5) + eps) + \
 np.log(norm.pdf(x[1], loc=prior_var_mu,
 scale=prior_var_sigma ** 0.5) + eps)

 return log_likelihood + prior

제안 분포로서 다변수 정규분포를 사용
def proposal_distribution(x):
 return norm.rvs(x[0]),
 chi2.rvs(df=1) * x[1]

수용 확률 계산
def acceptance_probability(x_new, x_current):
 # 로그확률을 사용하기 때문의 비율 연산이 - 연산이 되었습니다.
 # 이를 다시 일반 확률로 돌리기 위해 exp 함수를 적용합니다.
 return min(1,
 np.exp(target_log_distribution(x_new)
 - target_log_distribution(x_current)))

Metropolis-Hastings 알고리즘

```

```

def metropolis_hastings(iterations):
 samples = []
 x_current = [0, 1] # 초기값 설정

 for _ in range(iterations):
 # 새로운 제안 생성
 x_new = proposal_distribution(x_current)
 alpha = acceptance_probability(x_new, x_current)
 u = np.random.uniform(0, 1)

 # 수용 여부 결정
 if u < alpha:
 x_current = x_new

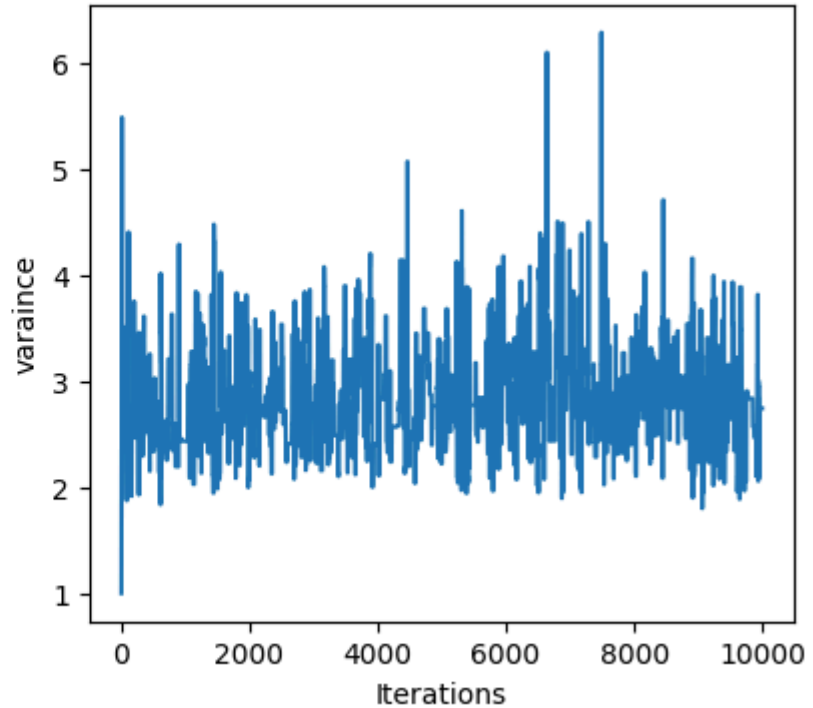
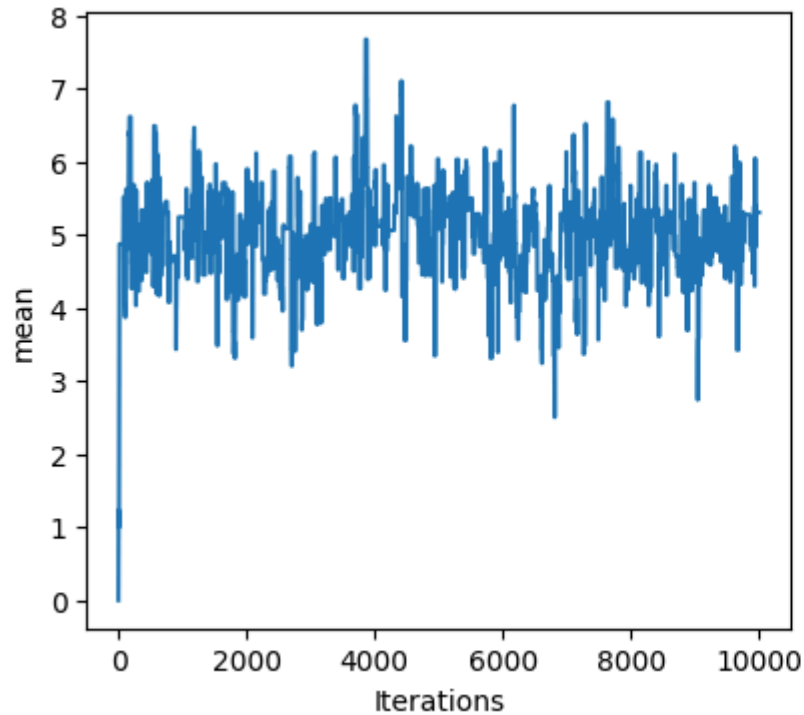
 # 샘플 저장
 samples.append(x_current)

 return np.vstack(samples)

Metropolis-Hastings 알고리즘 실행
X_sample = metropolis_hastings(10000)

결과 시각화
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for i, (ax, val) in enumerate(zip(axes, ['mean', 'varaince'])):
 ax.plot(np.arange(len(X_sample)) + 1, X_sample[:, i])
 ax.set_xlabel('Iterations')
 ax.set_ylabel(val)

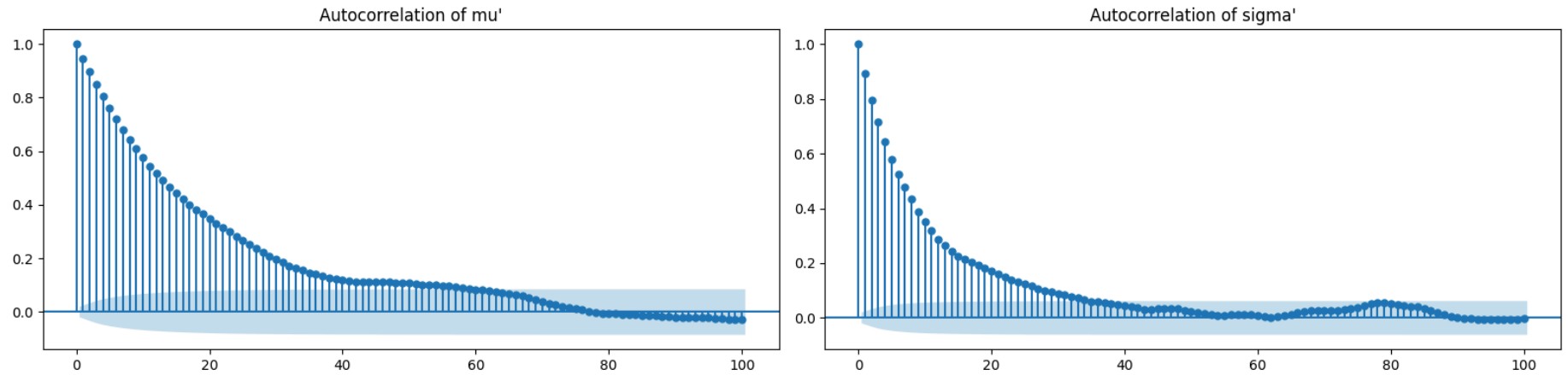
```



In [33]: `from statsmodels.graphics.tsaplots import plot_acf`

```
fig, axes = plt.subplots(1, 2, figsize=(16, 4))
plot_acf(X_sample[:, 0], lags=100, ax=axes[0])
axes[0].set_title("Autocorrelation of mu'")
plot_acf(X_sample[:, 1], lags=100, ax=axes[1])
axes[1].set_title("Autocorrelation of sigma'")

plt.tight_layout()
plt.show()
```



```
In [34]: X_samples_selected = X_sample[300::50]
```

```
In [35]: '''
코드는 Metropolis-Hastings 알고리즘을 통해 얻은
샘플들에 대한 자기 상관 함수(Autocorrelation Function, ACF)를
시각화합니다.

1. `fig, axes = plt.subplots(1, 2, figsize=(16, 4))`:
 1행 2열의 subplot을 생성합니다. 크기는 가로로 긴 형태이며,
 전체 figure의 크기는 가로 16인치, 세로 4인치입니다.
2. `plot_acf(X_sample[:, 0], lags=100, ax=axes[0])`:
 첫 번째 subplot에 대해서 Metropolis-Hastings 알고리즘을
 통해 얻은 샘플들의 첫 번째 요소에 대한 자기 상관 함수를
 계산하고 시각화합니다.
 `lags` 파라미터는 최대 시차를 설정하며,
 이 경우에는 100입니다.
3. `axes[0].set_title("Autocorrelation of mu'")`:
 첫 번째 subplot에 제목을 설정합니다.
 이 경우 "mu'에 대한 자기 상관 함수"라는 제목이 됩니다.
4. `plot_acf(X_sample[:, 1], lags=100, ax=axes[1])`:
 두 번째 subplot에 대해서 Metropolis-Hastings 알고리즘을 통해
 얻은 샘플들의 두 번째 요소에 대한 자기 상관 함수를
 계산하고 시각화합니다.
5. `axes[1].set_title("Autocorrelation of sigma'")`:
 두 번째 subplot에 제목을 설정합니다.
 이 경우 "sigma'에 대한 자기 상관 함수"라는 제목이 됩니다.
'''
```

```

6. `plt.tight_layout()` :
 subplot 간의 간격을 조정하여 레이아웃을 조밀하게 만듭니다.
 ...

추정된 평균과 표준편차
estimated_mean = np.mean(X_samples_selected, axis=0)[0]
estimated_std = np.mean(X_samples_selected, axis=0)[1]
print("Estimated Mean:", estimated_mean)
print("Estimated Standard Deviation:", estimated_std)

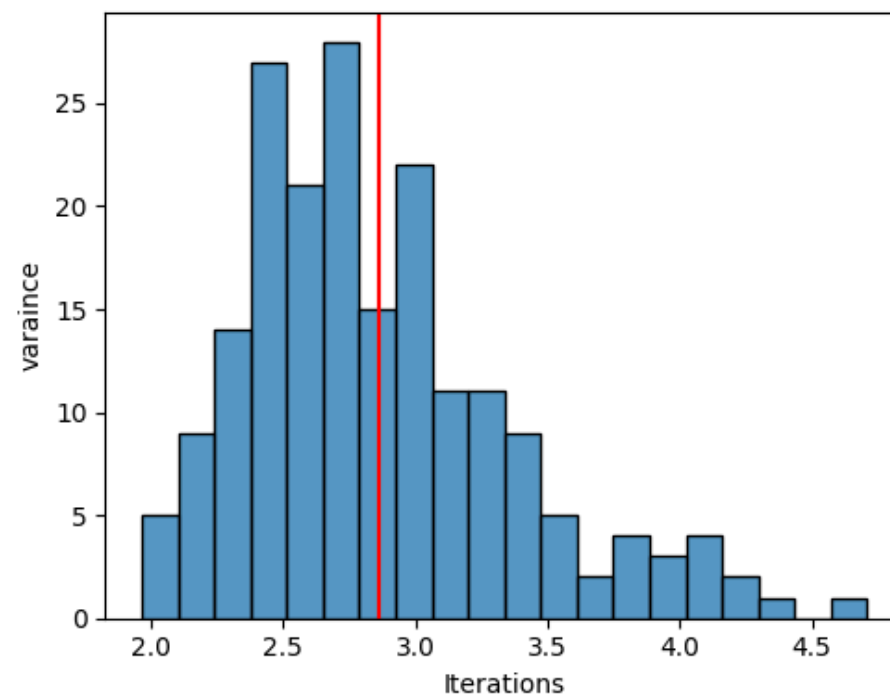
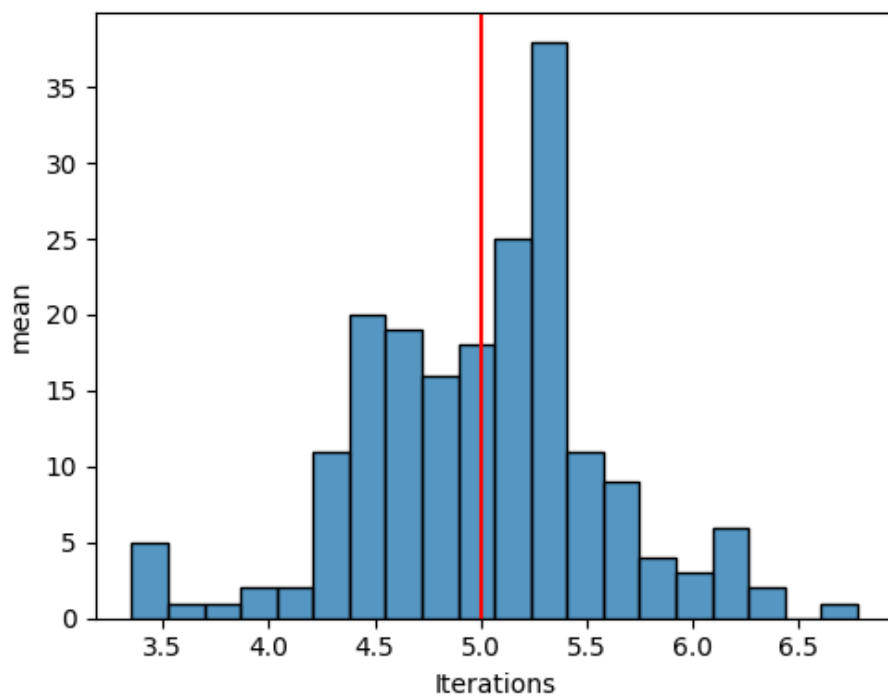
결과 시각화
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for i, (ax, val) in enumerate(zip(axes,
 ['mean', 'varaince'])):
 sns.histplot(X_samples_selected[:, i],
 ax=ax, bins=20)
 ax.set_xlabel('Iterations')
 ax.set_ylabel(val)
axes[0].axvline(estimated_mean, color='red')
axes[1].axvline(estimated_std, color='red')

plt.tight_layout()
plt.show()

```

Estimated Mean: 5.001567209293539

Estimated Standard Deviation: 2.8597940769334618



In [ ]: